INSTRUMENTATION & CONTROL — FINAL PROJECT

X-Wing VTOL Quadcopter

Data-Acquisition System & Autopilot Flight Controller

From Newton–Euler dynamics to a flying data-acquisition platform: cascaded PID control, a nine-state Extended Kalman Filter, six validated conditioning circuits, and ESP32 hardware.

9-state EKF	**6 circuits**	**RMSE 3.244 m**	**ESP32 + GUI**
sensor fusion	Proteus-validated	tuned outer loop	live telemetry

Amirkabir University of Technology · Department of Electrical Engineering

Presented to: Dr. Ahmad Afshar

Group members: [Amir Saeed Pashaei-Amir Ali Oliyaeemehr-Mohammad Amin Mir Kazemi]

ROADMAP

What we will cover

Nine chapters, three engineering phases — design, validation, implementation.

I Design & simulation

- Ch 1 — Modelling and equations of motion
- Ch 2 — Cascaded flight controllers
- Ch 3 — Sensing and the Extended Kalman Filter

II Electronics & validation

- Ch 4 — Six signal-conditioning circuits
- Ch 5 — Actuator selection and the mixer
- Ch 6 — Ground-control GUI

III Hardware & outlook

- Ch 7 — ESP32 hardware implementation
- Ch 8 — Innovation and commercialisation
- Ch 9 — Conclusions

The thread that runs through all nine chapters

A quadcopter has six degrees of freedom and only four actuators. Every design decision in this project — the cascade, the mixer, the estimator, even the choice of sensors — follows from that single structural fact.

01

Modelling the X-Wing Quadcopter

Reference frames, Newton–Euler dynamics, and control allocation

- Rotational kinematics and the Euler-angle rate transformation
- Translational and rotational equations of motion
- The mixer matrix: from motor speeds to thrust and torques
- Project scope, operational modes, safety, disturbances and faults

Reference frames and rotational kinematics

Two frames are used throughout: the earth-fixed inertial frame R, and the body-fixed frame R_3 with its origin at the centre of gravity.

- Rotation from inertial to body is defined by three successive Euler rotations — roll ϕ, pitch θ, yaw ψ.
- The gyroscope measures body rates p, q, r — these are NOT the Euler-angle derivatives, and the distinction matters.
- The transformation below is what links the two, and it reappears verbatim inside the EKF state-transition function in Chapter 3.

$$p = \dot{\phi} - \dot{\psi} \sin\theta$$
$$q = \dot{\theta} \cos\phi + \dot{\psi} \cos\theta \sin\phi$$
$$r = -\dot{\theta} \sin\phi + \dot{\psi} \cos\theta \cos\phi$$

Why not just integrate the gyro?

- Because the mapping is state-dependent — it contains ϕ and θ themselves.
- Integrating raw body rates as if they were Euler rates accumulates error immediately, and drifts without bound even in the ideal case.

Gimbal lock

- The transformation is singular at θ = ±90°.
- Documented as a safety requirement: quaternions are preferred over Euler angles for aggressive manoeuvres. Our flight envelope stays in the small-angle regime, so Euler angles are safe here.

Newton–Euler equations of motion

Translational — Newton's second law with gravity and thrust

$$X:\ mg\sin\theta = m[\,\dot{u} + wq - vr\,]$$
$$Y:\ -mg\cos\theta\sin\phi = m[\,\dot{v} - wp + ur\,]$$
$$Z:\ T - mg\cos\theta\cos\phi = m[\,\dot{w} + vp - uq\,]$$

Rotational — Euler's equations for a symmetric rigid body

$$\tau_\phi = I_{xx}\,\dot{p} + qr\,(I_{zz} - I_{yy})$$
$$\tau_\theta = I_{yy}\,\dot{q} + pr\,(I_{xx} - I_{zz})$$
$$\tau_\psi = I_{zz}\,\dot{r} + pq\,(I_{yy} - I_{xx})$$

The symmetry assumption

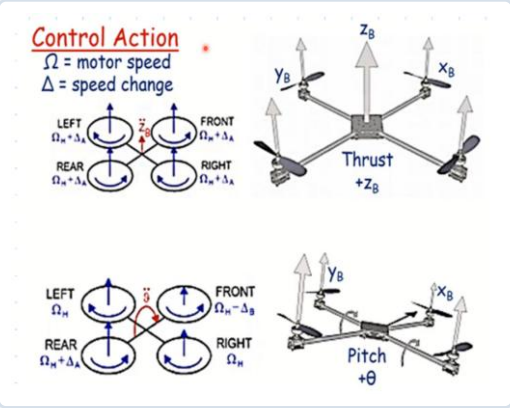
- The airframe is treated as symmetric, so the products of inertia vanish and the inertia matrix is diagonal: I = diag(I_xx, I_yy, I_zz).
- This is what lets the three rotational channels be designed independently.

Plant parameters used throughout

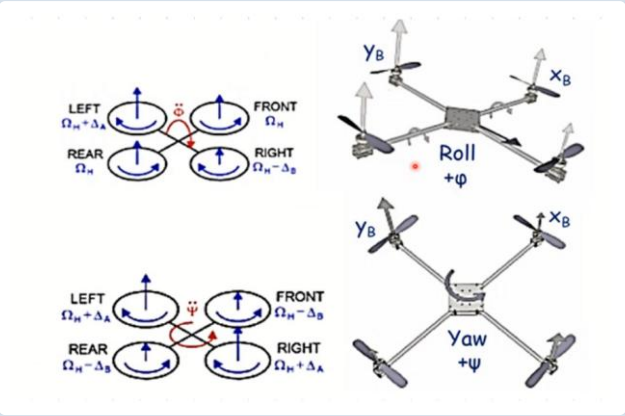
`m`	**1.2 kg**	vehicle mass
`I_xx , I_yy`	**0.01 kg·m²**	roll / pitch inertia
`I_zz`	**0.02 kg·m²**	yaw inertia
`g`	**9.81 m/s²**	gravity

How motor speeds produce motion

Standard cross (X-wing) configuration: motors 1 and 3 spin one way, 2 and 4 the other.



Thrust (U_1) and pitch torque (U_3) generation



Roll (U_2) and yaw (U_4) torque generation

U_1 — Altitude $U_1 = k_f(\omega_1^2+\omega_2^2+\omega_3^2+\omega_4^2)$

All four speeds increase proportionally; total thrust overcomes mg.

U_2 — Roll $U_2 = -l_y\ k_f(\omega_2^2 - \omega_4^2)$

Differential between the lateral pair only; front and rear lie on the axis and contribute nothing.

U_3 — Pitch $U_3 = -l_fx\ k_f\ \omega_1^2 + l_rx\ k_f\ \omega_3^2$

Differential between the longitudinal pair; the lateral pair contributes nothing.

U_4 — Yaw $U_4 = -k_m(\omega_1^2 - \omega_2^2 + \omega_3^2 - \omega_4^2)$

Exploits reactive drag torque: speed up the CW pair, slow the CCW pair. Net lift unchanged.

Control allocation — the mixer matrix

The abstraction that makes design possible

The controller never commands motor speeds. It commands four physically meaningful efforts: total thrust and three body torques. A fixed linear map converts them.

$$M = \begin{vmatrix} b & b & b & b \\ 0 & -bl & 0 & bl \\ -bl & 0 & bl & 0 \\ -d & d & -d & d \end{vmatrix}$$

$$[U_1\ U_2\ U_3\ U_4]^T = M \cdot [\omega_1{}^2\ \omega_2{}^2\ \omega_3{}^2\ \omega_4{}^2]^T$$

- **M is constant and invertible, so the required rotor speeds come back from a single inversion: $\omega^2 = M^{-1} U$.**
- In simulation the plant is driven directly in U-space, so M^{-1} is implicit.
- On hardware it becomes an explicit block sitting between the controller and the four ESCs.
- The controller therefore reasons in the intuitive space of newtons and newton-metres — never in rad/s.
- Note in the report: for a true Y4 coaxial-tail variant this matrix must be re-derived, and the lower coaxial rotor needs an aerodynamic correction factor for downwash.

Project scope and workflow

A continuous path from theoretical design to physical implementation.

1 Controller design
Newton–Euler model; cascaded inner attitude loop and outer position loop

2 Simulate
Verify pole placement against overshoot and peak-time targets

3 Add sensors
Mathematical sensor models inject real noise, bias and sample rates

4 Simulate again
Evaluate filtering and control robustness under realistic measurement

5 Conditioning circuits
Filters, amplifiers, level shifters — designed from specification

6 Simulate in Proteus
Electrical and protocol validation before any fabrication

7 Hardware design
Microcontroller, sensor and actuator interface selection

8 Hardware implementation
Physical assembly, integration and functional DAQ testing

Operational modes and operator interaction

Manual / Remote

- Operator sends roll, pitch, yaw and throttle directly by RC.
- Inner loop stabilises attitude; the outer position loop is closed by the human.
- Requires visual line of sight. Operator cognitive load is at its peak.

Automatic / Autonomous

- Autopilot has full authority over both loops.
- Waypoint navigation, position hold, and DAQ executed with no stick input.
- Operator shifts to a supervisory role — monitoring battery, link integrity and sensor health.

Local / GCS

- Supervisory commands over the telemetry datalink.
- Return-to-launch, auto-land, and in-flight mission modification.
- Mediated through the ground-station GUI built in Chapter 6.

The most critical operator responsibility

In automatic mode the operator is not passive. Their job is maintaining situational awareness so that a manual takeover — the fail-safe override — can be executed the moment a software anomaly or a positional sensor failure such as GPS degradation occurs. The autonomy is trusted, but it is supervised.

Safety and security requirements

Safety — preventing physical and systematic failure

- Sensor health & redundancy — pre-flight noise and calibration checks; estimators must reject invalid data.
- Frame conversion — strict validation of ground-to-body transforms; quaternions preferred to avoid gimbal lock.
- Command limitation & anti-windup — controller outputs saturated before the mixer. Saturated motors draw stall currents, overheat ESCs, and cost degrees of freedom.
- Motor & propeller health — verified rotation direction, dynamic balance, frame joint integrity.
- Power & energy management — supervisory monitoring of state of charge, current and temperature; RTL energy computed against distance and wind.
- Environmental limits and emergency-state logic — RTL on signal loss, motor kill-switch on unrecoverable flip.

Security — protecting the link and the software

Data link protection

Encrypted, authenticated telemetry (e.g. signed MAVLink) to prevent eavesdropping and fake command injection.

Access control

Controller gains and limits restricted to authorised users — configuration is an attack surface.

Computational stability

- Code must survive invalid data, noise and numerical error.
- Watchdog timers autonomously reset the microcontroller on a software freeze.

Expected disturbances and their prioritisation

Priority	Disturbances in this class	Required system response
HIGH — Critical	Motor or propeller failure; severe voltage drop; loss of a main sensor; frame-conversion error; loop instability	Immediate emergency termination — motor kill-switch, parachute deployment
MEDIUM — Degradation	Sensor noise; slight actuation delay; moderate wind	Compensate through robust filtering; autonomous return-to-launch if positional sensing is compromised
LOW — Nuisance	UI delays; logging errors; short transient oscillation	Corrected by the feedback loop itself; no threat to flight safety

Where disturbances enter

- Sensors — high-frequency noise, gyro drift, feedback loss, hard/soft-iron miscalibration
- Coordinate conversion — incorrect angles or matrix misalignment when projecting between frames
- Environment and power — wind, turbulence, EMI, voltage sag, overheating

The design response

- EKF for sensor fusion — attenuates high-frequency accelerometer vibration and low-frequency gyro drift simultaneously
- Innovation-covariance outlier rejection within the filter
- Software IIR/FIR low-pass on raw IMU signals; derivative-gain tuning widens inner-loop bandwidth for wind rejection

Foreseeable operational faults and responses

Actuator faults

- A quadcopter cannot survive complete motor or ESC loss — there are exactly four actuators and no spare control authority.
- Front arm failure: torque symmetry is lost instantly and the vehicle flips. Response is IMU-based detection, motor kill, parachute.
- Coaxial tail analytical redundancy: if one of two coaxial tail motors fails, the survivor still generates lift; the FTC algorithm reconfigures the mixer and uses excess front-motor torque to counter the induced yaw spin.

Sensor & navigation failures

- GPS loss or high HDOP: exit absolute position tracking, fuse IMU and barometer for short-term dead reckoning, hold a safe hover until recovery or auto-land.
- Magnetometer corruption from EMI: the EKF drives the magnetometer measurement weight to zero and falls back on gyro integration for yaw.

Communication & power failures

- Telemetry or RC loss triggers the fail-safe protocol: adjust altitude and execute return-to-launch autonomously.
- Critical low battery overrides the current mission — return-to-home or immediate auto-land depending on altitude and distance.

02

Design of the Flight Controllers

A two-level cascade designed by pole placement, tuned by RMSE

- Why under-actuation forces a cascaded architecture
- Inner loop — attitude and altitude by PD pole placement
- Outer loop — position, the R(ψ) rotation, and the upgrade to PID
- Wind disturbance rejection and square-trajectory benchmarking

The central problem: under-actuation

6

degrees of freedom
(3 translational, 3 rotational)

vs

4

independent actuators
(the four rotor speeds)

- **Horizontal motion cannot be commanded directly. It exists only as a by-product of tilting the airframe so that a component of rotor thrust acts horizontally.**
- That physical coupling is the single fact that shapes the entire architecture.
- A monolithic controller mapping trajectory straight to rotor speeds is intractable: the speeds enter nonlinearly, and every output depends on all four inputs at once.

Two decompositions make it tractable

- Control allocation — the mixer separates physically meaningful efforts from individual motor commands.
- Cascade — a fast inner attitude loop nested inside a slow outer position loop.

$$\ddot{x} = (u_1/m)(\cos\phi \sin\theta \cos\psi + \sin\phi \sin\psi)$$
$$\ddot{y} = (u_1/m)(\cos\phi \sin\theta \sin\psi - \sin\phi \cos\psi)$$

Supplying ϕ_{ref} and ϑ_{ref} by hand to reach a position is impractical — the mapping is nonlinear and coupled to yaw. The outer loop exists precisely to close this gap.

The cascaded control architecture

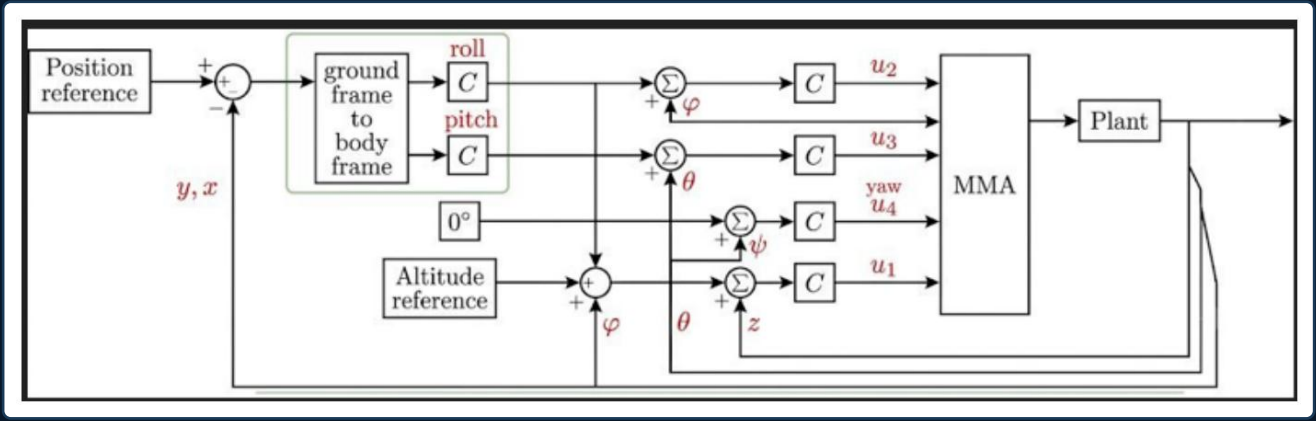


Figure 2.1 — the complete control architecture

mission references → outer loop → attitude references → inner loop → control efforts → mixer → rotor speeds → plant → state

Block	Inputs	Outputs
Outer loop (position)	x_ref, y_ref (global); measured x, y, ψ	ϕ_ref, θ_ref
Inner loop (attitude/altitude)	ϕ_ref, θ_ref, ψ_ref, z_ref; measured state	u_1, u_2, u_3, u_4
Motor mixer	u_1, u_2, u_3, u_4	ω_1^2, ω_2^2, ω_3^2, ω_4^2
Plant (Newton–Euler)	rotor speeds via thrust and torques	12-state vector
Overall system	x_ref, y_ref, z_ref, ψ_ref	x, y, z, ψ and the full state

Inner-loop design by pole placement

- **Under the hover linearisation (small angles, p ≈ q ≈ r ≈ 0) the four channels decouple. Each becomes an independent double integrator.**

$$\ddot{\phi} = u_2 / I_{xx} \quad \Longrightarrow \quad G(s) = 1 / (I_{xx}\, s^2)$$

$$C(s) = K_d\, s + K_p \qquad T(s) = (K_d\, s + K_p) / (I_{xx}\, s^2 + K_d\, s + K_p)$$

$$K_p = \omega_n^2\, I_{xx} \qquad K_d = 2\zeta\, \omega_n\, I_{xx}$$

- Specification: $T_p = 0.3$ s and P.O. = 5 % $\Longrightarrow$ $\zeta = 0.7$, $\omega_n = 14.7$ rad/s, poles at $s = -10.29 \pm j10.50$
- Roll and pitch: $K_p = (14.7)^2(0.01) = 2.16$, $K_d = 2(0.7)(14.7)(0.01) = 0.21$
- Yaw uses $I_{zz} = 0.02$, which doubles both gains exactly
- Altitude is linearised about a non-zero operating point: $\delta\ddot{z} = \delta u_1/m$, with a hover feed-forward $u_1 eq = mg$ added so the controller only regulates deviation from hover thrust

Channel	Plant	ω_n	ζ	K_p	K_d
Roll (ϕ)	1/(I_xx s²)	14.7	0.7	2.16	0.21
Pitch (θ)	1/(I_yy s²)	14.7	0.7	2.16	0.21
Yaw (ψ)	1/(I_zz s²)	14.7	0.7	4.32	0.42
Altitude (z)	1/(m s²)	4.4	0.7	23.2	7.4

Why altitude is slower

A smoother response is chosen deliberately: $T_p = 1$ s, P.O. = 5 %, giving $\omega_n = 4.4$ rad/s. Altitude does not need attitude-level bandwidth, and a gentler vertical response is easier on the propulsion system.

Derivative on measurement, not error

$u_2 = K_p(\phi_{ref} - \phi) - K_d \cdot \dot{\phi}$ — the reference is never differentiated; the gyro rate is used directly.

Implementation in Simulink

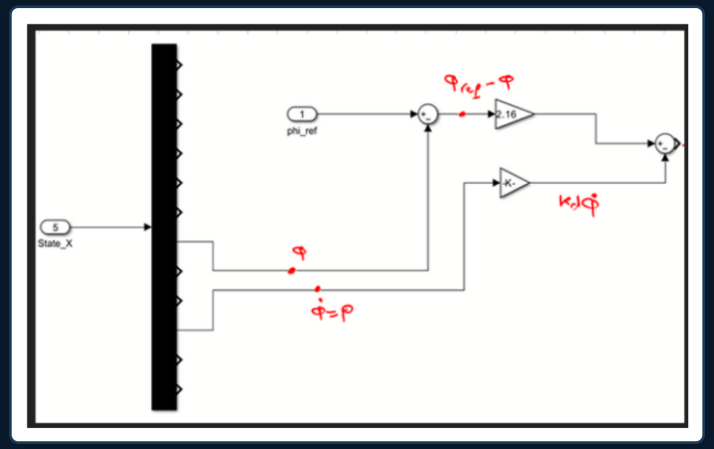


Figure 2.2 — single PD channel: roll error × K_p, measured rate × K_d, combined into u_2

- Each PD controller is realised from primitive blocks — no built-in PID block, so every gain is visible and traceable.
- The error (ϕ_ref − ϕ) is scaled by K_p; the measured body rate is scaled by K_d; the two are subtracted.
- Pitch, yaw and altitude replicate the identical structure, fed by the reference inputs and the State_X bus.
- The four efforts are multiplexed into U_vector = [u_1, u_2, u_3, u_4] and handed to the plant.

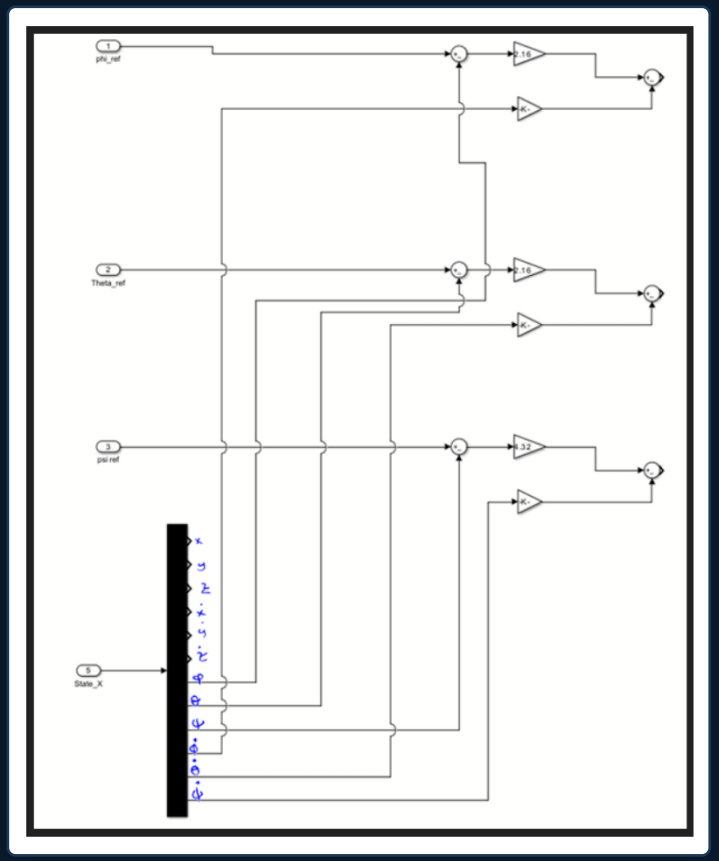


Figure 2.3 — the complete inner-loop controller: four PD channels multiplexed into U_vector

Inner-loop simulation results

Design targets met on both channels — the pole placement is confirmed.

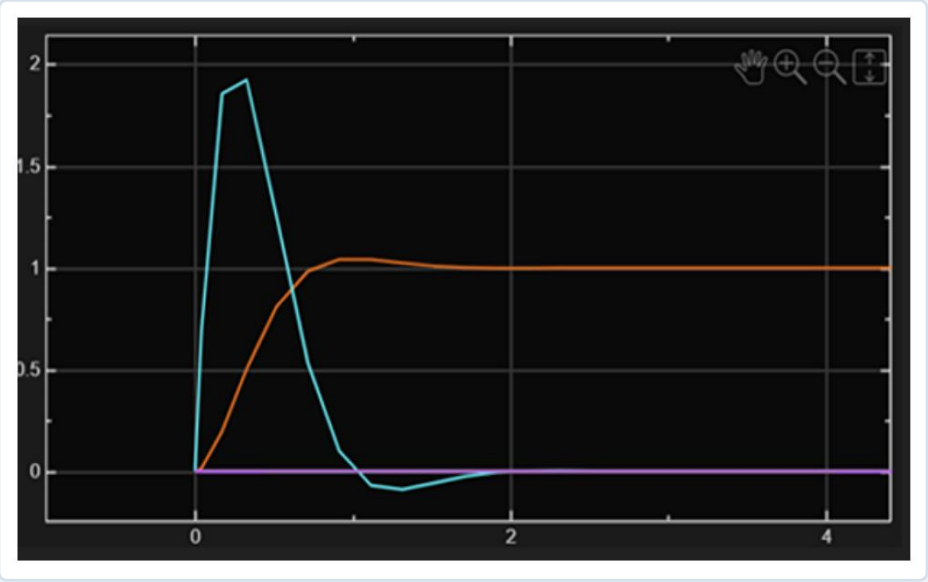


Figure 2.4 — altitude step to z_ref = 1 m

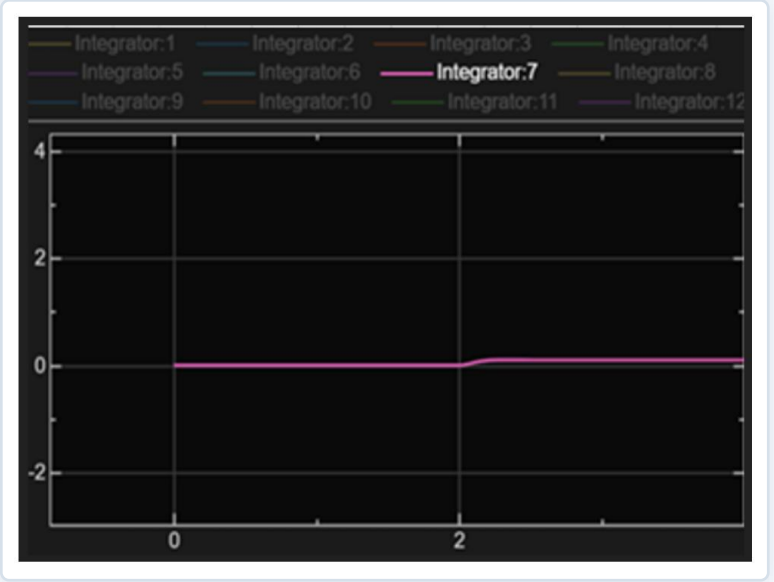


Figure 2.5 — roll step to ϕ_ref = 0.1 rad at t = 2 s

Channel	Measured P.O.	Target P.O.	Measured T_p	Target T_p
Altitude	4.1 %	5 %	0.918 s	1 s
Roll	5.0 %	5 %	0.29 s	0.3 s

Because roll, pitch and yaw share the same controller form, testing roll alone is sufficient to validate all three.

Outer-loop design and the R(ψ) rotation

- **Treat the inner loop as an ideal, fast tilt-tracking element. With hover thrust, small angles and ψ = 0, the horizontal dynamics reduce to ẍ = gθ and ÿ = −gϕ.**

$$[\theta_{des} ; \phi_{des}] = (1/g) \cdot R(\psi) \cdot [\ddot{x}_{des} ; \ddot{y}_{des}]$$

$$R(\psi) = [\ cos\ \psi \quad sin\ \psi\ ;\ sin\ \psi \quad -cos\ \psi\ ]$$

- The g produced by the plant cancels the 1/g inside the rotation block, so the effective plant seen by the outer compensator is a pure double integrator, G(s) = $1/s^2$.
- To preserve time-scale separation a slow, well-damped response is chosen: ζ = 0.9, ω_n = 2 rad/s ⟹ K_p = 4, K_d = 3.6.
- Saturation blocks limit commanded tilt, keeping the vehicle inside the small-angle regime the design assumes.

What the rotation block actually does

- It does NOT translate the position reference into a body-frame position.
- It rotates the horizontal acceleration command by the current heading ψ, so that the resulting roll and pitch set-points point the vehicle correctly regardless of which way it faces.
- At ψ = 0 the rotation reduces to the identity — which is why heading is held at zero in the nominal tests.

Why the outer loop must exist

- A pilot or mission planner specifies where the vehicle should be, not the precise tilt angles needed to get there.
- The outer loop generates the attitude set-points a pilot would otherwise have to guess.

Implementation and circular-trajectory validation

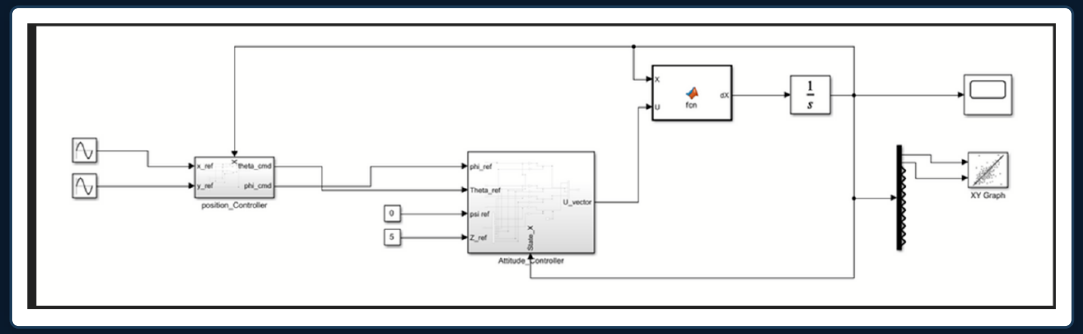


Figure 2.6 — the full cascaded model

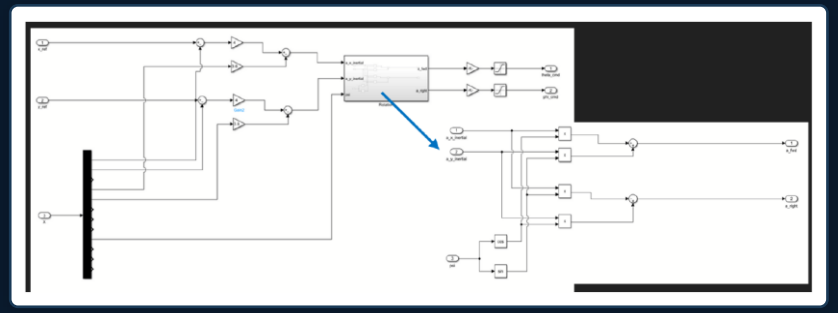


Figure 2.7 — inside the position controller and the R(ψ) block

$$x_ref = 5\sin(2\pi \cdot 0.5 \cdot t) \quad y_ref = 5\cos(2\pi \cdot 0.5 \cdot t)$$

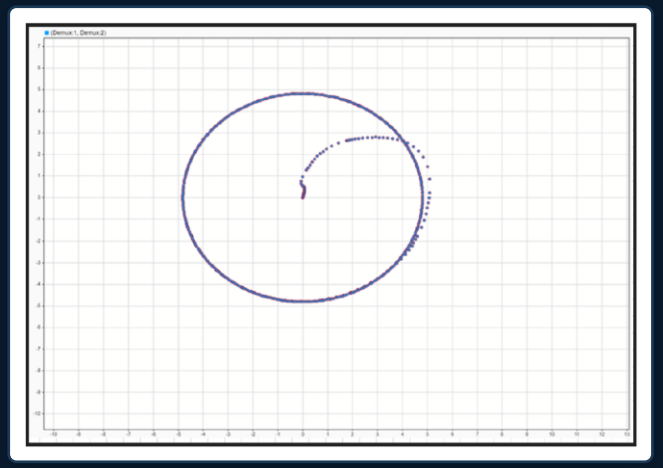


Figure 2.8 — circular tracking, radius 5 m

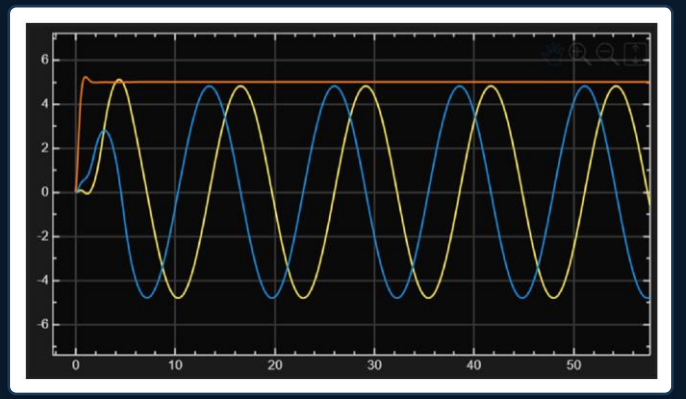


Figure 2.9 — x(t) and y(t): quadrature sinusoids of amplitude 5 m

Wind disturbance and the upgrade to PID

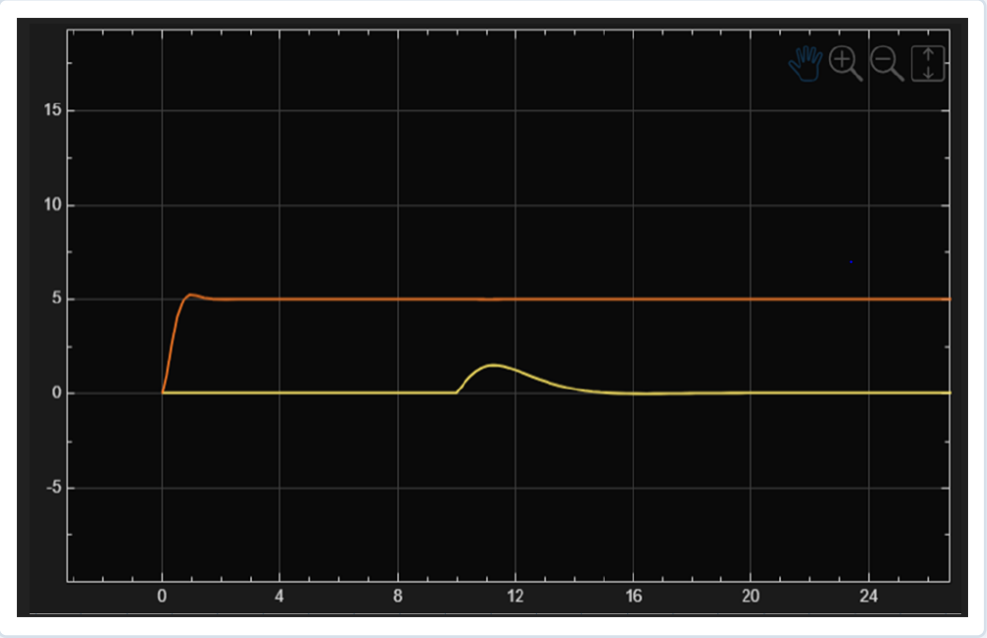


Figure 2.10 — PID outer loop; x disturbed at t ≈ 10 s and driven back to reference

- **A constant 2 m/s² horizontal acceleration is injected on the x channel to represent steady wind.**
- With a PD outer loop the drone drifts and holds at roughly x = 2 m. This is expected, not a bug: a PD controller cannot reject a constant disturbance without a residual error, because a non-zero error is required to generate the opposing effort.

$$u = K_p\, e + K_d\, \dot{e} + K_I \int e\, dt$$

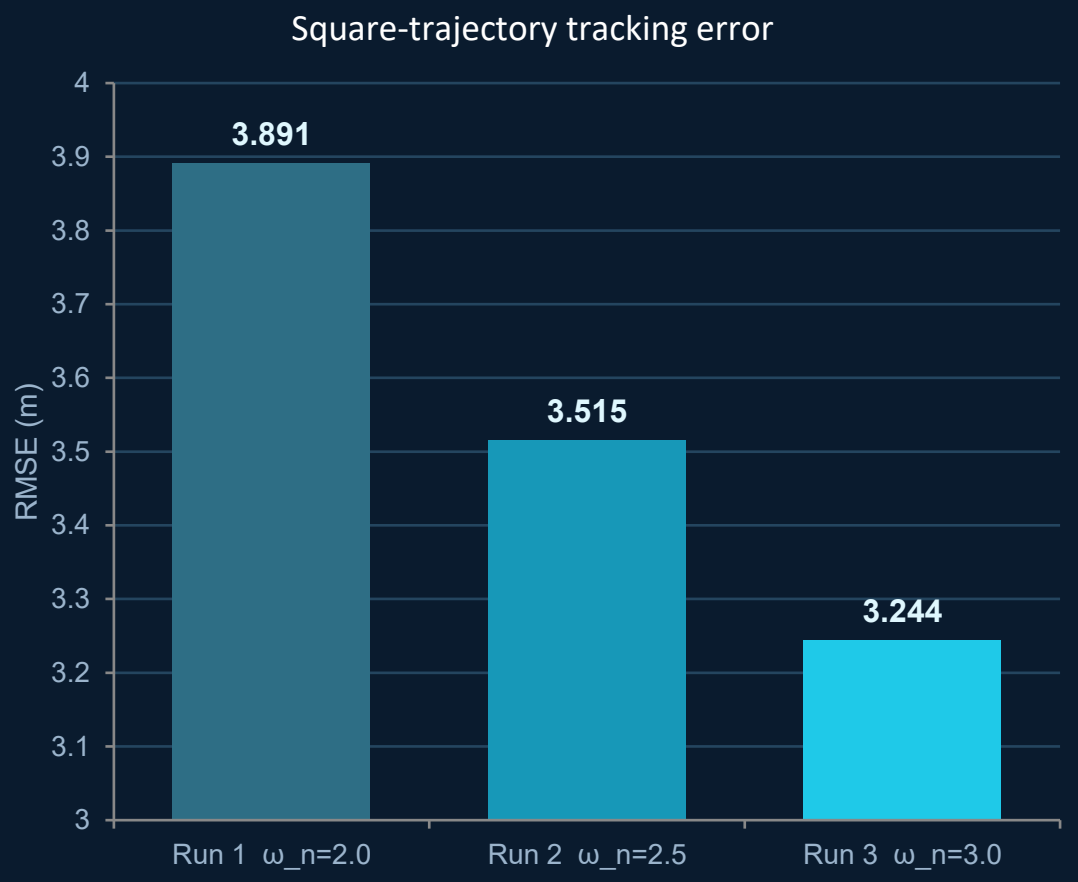
- The integrator accumulates the standing error until the control effort exactly cancels the disturbance, driving steady-state error to zero. K_I = 2 gave the fastest well-behaved recovery.
- **Sweeping the disturbance magnitude showed the vehicle rejects horizontal accelerations up to about 4 m/s².**
- That ceiling is set by the tilt-saturation limit, a_max = g·tan θ_sat — not by the integral action. Raising K_I further would not help.

Final benchmark — square-trajectory tuning by RMSE

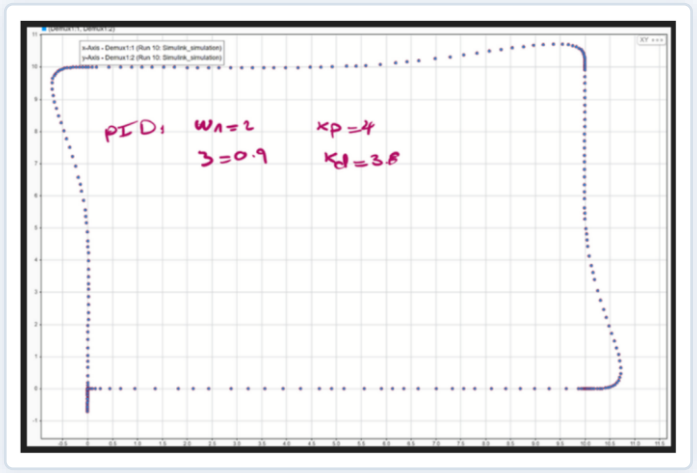
- The square is the harder test: its corners are velocity discontinuities that no smooth linear controller can follow without overshoot.
- RMSE is therefore dominated by corner behaviour, making it a sensitive discriminator between gain sets.
- Three PID designs were compared, each parameterised by ω_n and ζ mapping to $K_p = \omega_n^2$ and $K_d = 2\zeta\omega_n$, with $K_I = 2$ held fixed.

Design	ω_n	ζ	K_p	K_d	K_I	RMSE (m)
Run 1	2.0	0.9	4.00	3.6	2	3.891
Run 2	2.5	0.9	6.25	4.5	2	3.515
Run 3 (final)	3.0	0.9	9.00	5.4	2	**3.244**

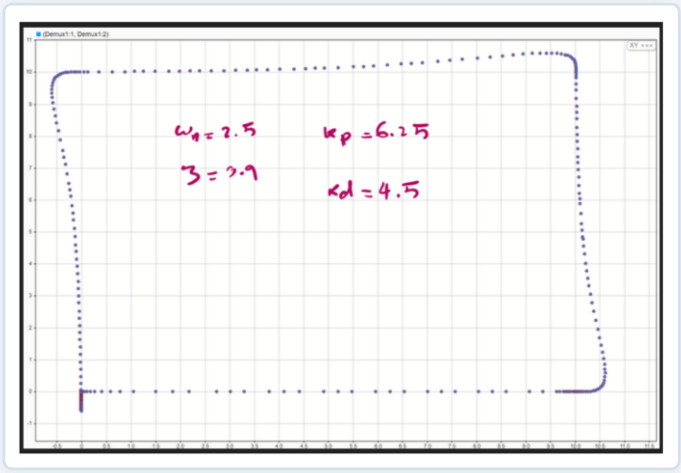
RMSE decreases monotonically as outer-loop bandwidth is raised. Final design: $K_p = 9$, $K_d = 5.4$, $K_I = 2$.



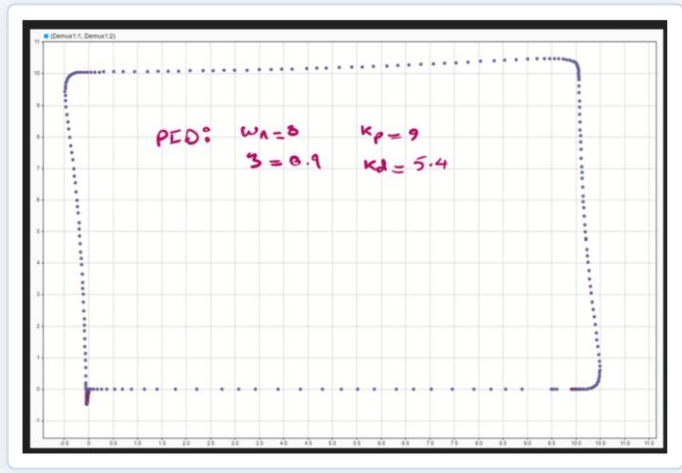
Square-trajectory tracking across the three gain sets



Run 1 — K_p = 4.00, RMSE 3.891 m



Run 2 — K_p = 6.25, RMSE 3.515 m



Run 3 — K_p = 9.00, RMSE 3.244 m

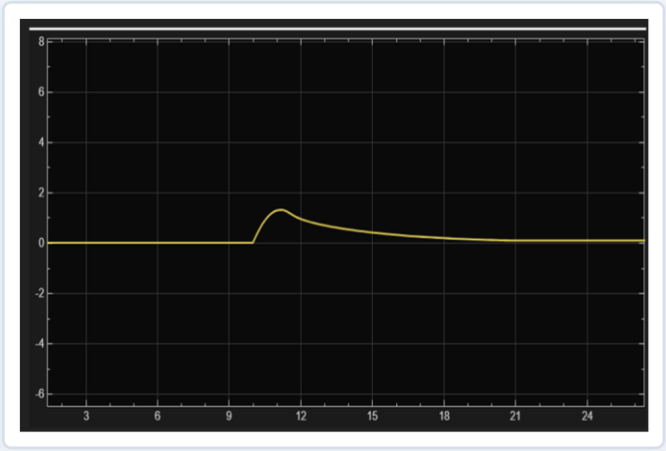


Figure 2.12 — wind robustness re-verified with the final coefficients

Robustness is not traded away for speed

- The wind test was repeated with K_p = 9, K_d = 5.4, K_I = 2.
- The disturbance is again rejected to zero steady-state error — the faster gains do not compromise disturbance rejection.
- This matters: a common failure mode in tuning is buying tracking performance at the cost of robustness. Here we verified explicitly that it did not happen.

CH 2 · SUMMARY

Chapter 2 — what was established

1 Motor mixing

Decouples intuitive thrust and torque efforts from individual rotor commands via a constant, invertible matrix.

2 Inner loop

PD by pole placement: roll/pitch 2.16 + 0.21s, yaw 4.32 + 0.42s, altitude 23.2 + 7.4s with hover feed-forward. 5 % overshoot and peak-time targets met.

3 Outer loop

Converts a global horizontal-acceleration demand — rotated into body reference by $(1/g)\cdot R(\psi)$ — into tilt set-points. Validated on circular tracking.

4 Disturbance rejection

Wind motivated the upgrade from PD to PID; square-trajectory RMSE tuning fixed the gains at $K_p = 9$, $K_d = 5.4$, $K_I = 2$.

These controllers assume a perfect, noise-free state. On real hardware that state does not exist — which is Chapter 3.

03

Sensing and State Estimation

Selecting the suite, modelling it honestly, and recovering the state with an EKF

- Five sensors — GPS, IMU, barometer, rangefinder, magnetometer
- Redundancy: which quantities are over-determined, and which cannot be
- The two deficiencies that make an estimator unavoidable
- A nine-state Extended Kalman Filter, and what it recovers

The sensor suite

The outer loop needs horizontal position; the inner loop needs attitude, rate and altitude.

Sensor	Model	Measures	Full-scale range	Interface
GPS	u-blox NEO-6M	x, y and velocity	global; ≤ 50 km altitude	UART / NMEA
IMU	InvenSense MPU6050	acceleration, angular rate	±4 g / ±500 °/s	I²C
Barometer	MEAS MS5611	pressure → altitude	10–1200 mbar	I²C / SPI
Rangefinder	MB1240 → GP2Y0A21YK	height above ground	20–200 cm → 10–80 cm	analog
Magnetometer	Honeywell HMC5883L	magnetic field → yaw	±1.3 gauss	I²C

Chosen for cost, not performance

Every part sits at the low-cost end. The NEO-6M is modest at 1–5 Hz; the MPU6050 drifts and is vibration-sensitive. That trade is acceptable precisely because the EKF corrects those deficiencies.

Each has an uncorrelated failure mode

Weather for the barometer, sky view for GPS, integration drift for the IMU, motor EMI for the magnetometer. Independence is what makes fusion robust.

One deliberate gap

No sensor in this list measures roll or pitch. That is not an oversight — no affordable sensor does. It is the reason the estimator exists.

GPS and IMU — the two anchors of the estimator

G u-blox NEO-6M — absolute position

- Trilaterates orbiting satellite signals to give absolute position and velocity — the only non-drifting reference in the suite.
- ≈2.5 m CEP horizontal accuracy; ≈0.1 m/s velocity accuracy; 1–5 Hz update over UART/NMEA.
- Range is effectively global, with a 50 km altitude ceiling and ~500 m/s velocity limit — so it never saturates on our trajectories.
- Weaknesses: slow, poor with limited sky view, degrades during fast manoeuvres, and its vertical channel is markedly worse than its horizontal one.
- Chosen because it is cheap, trivially integrated, and entirely sufficient once corrected by higher-rate inertial data.

I InvenSense MPU6050 — the driving input

- 3-axis accelerometer (specific force) plus 3-axis gyroscope (angular rate), both body-frame, both fast.
- Selectable ±2/4/8/16 g and ±250/500/1000/2000 °/s; sampled up to 1 kHz. We use ±4 g and ±500 °/s.
- Weaknesses that shape the whole chapter: no internal magnetometer, so no absolute heading; gyroscope drift; sensitivity to motor and propeller vibration; no position or altitude at all.
- It is the backbone of the fast inner loop and — crucially — the input that drives the EKF prediction step.

Fast, drifting, relative (IMU) + slow, stable, absolute (GPS) = the classic strapdown INS/GPS pairing

Barometer, rangefinder — and an honest substitution

MEAS MS5611 barometer

- Infers altitude from static pressure; 10–1200 mbar covering roughly −500 m to +9000 m, with resolution as fine as ~10 cm.
- Weaknesses are environmental, not intrinsic: drifts with weather, disturbed by propeller wash, filtering adds delay, and it cannot measure true distance to the ground.
- Below ≈1 m, ground effect and propeller-induced pressure fluctuations corrupt the reading — precisely during landing.

The specified sensor: MaxBotix MB1240

- Ultrasonic time-of-flight; analog output at Vcc/1024 per cm, 20–200 cm range, 10 Hz, narrow beam suited to a downward altimeter.
- Chosen to complement the barometer exactly where the barometer fails, giving a third independent altitude source with a different failure mode.
- It could not be sourced from the local market.

The substitute: Sharp GP2Y0A21YK

- Infrared triangulation — distance from the parallax angle of a reflected beam, output as an analog voltage.
- Range only 10–80 cm; update period ~38 ms.
- Most importantly: a nonlinear, non-monotonic transfer curve. Output rises as objects approach but folds back below ~10 cm, making very short distances genuinely ambiguous.
- Consequence: part of the conditioning burden moves into firmware, and the fold-back needs a guard near the ground.

Distance	V_out	Distance	V_out
10 cm	2.30 V	40 cm	0.70 V
15 cm	1.65 V	50 cm	0.58 V
20 cm	1.30 V	60 cm	0.50 V
30 cm	0.90 V	80 cm	0.40 V

```
d = 27.86·V^−1.15
                −
            0.42
```

Datasheet curve fitted in firmware

Magnetometer and the redundancy scheme

Honeywell HMC5883L

- 3-axis digital magnetometer; ±0.88 to ±8.1 gauss selectable — the ±1.3 gauss setting comfortably contains the Earth's 0.25–0.65 gauss field. Heading resolution 1–2°.
- It exists for one essential reason: the gyroscope yields yaw only by integration, which drifts, and yaw is weakly observable from GPS.
- Highly sensitive to motor, battery and ESC currents; needs hard- and soft-iron calibration; too slow and noisy for the fast inner loop alone.

The single point of failure we cannot remove

- Redundancy covers sensing, not actuation.
- A quadcopter cannot survive complete motor or ESC failure — exactly four actuators, no spare control authority. True actuator redundancy needs a hexacopter or octocopter.
- This is an architectural limit of the airframe, and no sensor suite can remedy it.

Altitude redundancy

Over-determined by three independent sources — MS5611 barometer, GPS altitude, IMU vertical acceleration — fused in the EKF, with the rangefinder covering the sub-1 m regime. Their failure modes are uncorrelated, so any one degrading is survivable.

Attitude redundancy

Reconstructed from three sources: the gyroscope (fast but drifting), the accelerometer (senses the gravity direction, so bounds roll and pitch), and the magnetometer (fixes absolute yaw). No single one suffices; together they span the attitude state.

Navigation redundancy

GPS is backed by short-term IMU/barometer dead reckoning — long enough to trigger a fail-safe. Loss of GPS initiates hover or landing; low battery initiates return-to-home.

The Simulink sensor subsystem

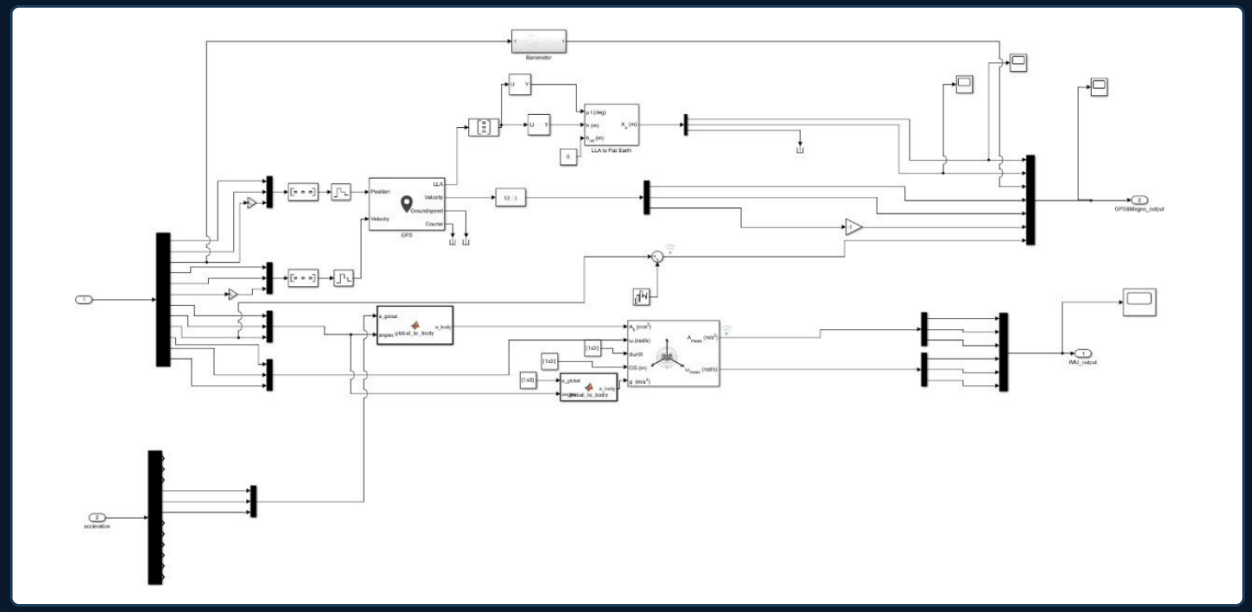


Figure 3.1 — the true state is distributed to the GPS, barometer, IMU and magnetometer models

- **This subsystem models sensors — it does not estimate.**
- Its purpose is to degrade the perfect state into measurements carrying the correct noise, bias, quantisation, coordinate frames and sample rates that each real device would impose.
- Input 1 is the full true state from the plant. Input 2 carries true linear acceleration, which the accelerometer model needs but which is not part of the integrated state vector.
- A bank of Demux and Selector blocks fans the vectors into individual components so each sensor model receives exactly the quantities it measures — routing only, no computation.

The IMU path — two rotations before the block

- The accelerometer does not measure world-frame kinematic acceleration — it measures body-frame specific force.
- Two global_to_body MATLAB Function blocks therefore rotate the true acceleration and gravity into the body frame before the IMU block, so specific force = kinematic acceleration − gravity is formed correctly.

The magnetometer path — a deliberate simplification

- True yaw is taken from the state and band-limited white noise is added at a summing junction.
- This captures the essential character of a real compass — an absolute heading corrupted by noise — without modelling the full three-axis magnetic field.

The GPS path and the LLA ↔ flat-Earth round trip

The most intricate part of the subsystem — and a deliberate one.

1 Down-sample and reframe

A −1 gain flips the vertical components: the plant works z-up (ENU family), the GPS block expects North-East-Down. A Zero-Order Hold samples down to the 10 Hz GPS rate — the block rejects continuous-time inputs anyway.

2 The GPS block outputs LLA

It adds position and velocity noise with the slow, correlated random-walk character of a real receiver, and emits Latitude, Longitude, Altitude — geodetic angles on the ellipsoid, not local metres.

3 Why conversion is mandatory

The controller, estimator and plant all reason in local Cartesian metres. You cannot subtract a reference position from a latitude in degrees and get a metre-valued error.

4 LLA to Flat Earth

Selector blocks split the vector into a 2×1 lat/lon port, a scalar altitude port, and a reference altitude defining the tangent-plane origin. Output X_e is a 3×1 position in metres — again NED, so a second −1 gain returns it to z-up.

true metres → (into GPS) → noisy LLA → (out of GPS) → noisy metres. The round trip is deliberate: it is precisely how the noise a real receiver introduces in the geodetic domain gets injected into the local position estimate.

Two measurement buses — and two problems

Output bus	Elements	Source	Rate
IMU_output	a_x, a_y, a_z, p, q, r	IMU block (A_meas, ω_meas)	0.005 s — 200 Hz
GPS&Magno_output	x, y, z, v_x, v_y, v_z, yaw	GPS (x, y, v), barometer (z), magnetometer (yaw)	0.1 s — 10 Hz

The multirate structure is deliberate and physically faithful: inertial measurements are fast, GPS-derived measurements are slow. This is exactly the fast-IMU / slow-GPS pattern the EKF is built around — predict at the IMU rate, correct at the GPS rate — and it is established here, at the sensor level, so the estimator inherits it directly.

1 No direct sensor for roll and pitch

- The gyroscope measures rate, not angle — integrating it drifts without bound.
- The accelerometer senses the gravity vector, from which tilt can be inferred, but only when the vehicle is not accelerating. In flight its output is a mixture of gravity and manoeuvre acceleration.
- The magnetometer fixes only yaw. So roll and pitch are unmeasured states that must be estimated.

2 Every measurement is noisy

- GPS position carries metre-level noise; the accelerometer is polluted by motor and propeller vibration; the magnetometer is disturbed by motor and ESC currents; the barometer fluctuates with propeller wash.
- In simulation the raw GPS position spans roughly ±10 m about a true value of a few metres.
- Feeding that straight to a controller designed for the clean state injects the noise directly into the motor commands.

The Kalman filter — predict, correct, and the two covariances

Predict — advance the state, grow the uncertainty

$$\hat{x} \leftarrow F\,\hat{x} + B\,u \qquad P \leftarrow F\,P\,F^{T} + Q$$

Update — weigh the measurement, shrink the uncertainty

$$K = P\,H^{T}\,(H\,P\,H^{T} + R)^{-1}$$
$$\hat{x} \leftarrow \hat{x} + K\,(z - H\,\hat{x})$$
$$P \leftarrow (I - K\,H)\,P$$

- **The gain K is simply a trust weighting. Large R (noisy sensor) drives K towards zero so the measurement is largely ignored; small R lets the measurement dominate.**

Extension to a nonlinear plant

- Constant F and H do not exist for a quadcopter. The EKF keeps the full nonlinear functions to propagate — $\hat{x} \leftarrow f(\hat{x},u)$, $\hat{z} = h(\hat{x})$ — and linearises only for the covariance arithmetic, using Jacobians $F = \partial f/\partial x$ and $H = \partial h/\partial x$ at the current estimate. Every other equation is identical.

R — measurement-noise covariance

- Diagonal entries are the variances of each sensor's noise. It answers: how much do I distrust each measurement?
- Read essentially from the datasheets — a GPS with σ = 2.5 m contributes 6.25 to the corresponding diagonal.

Q — process-noise covariance

- Captures everything the prediction model fails to represent: un-modelled dynamics, discretisation error, and the noise entering through the inertial inputs. It answers: how much do I distrust my own model?
- Unlike R it cannot be read from a datasheet — it is seeded from inertial noise densities and then tuned. The ratio Q/R sets the filter's character: large Q/R tracks quickly but noisily; small Q/R smooths heavily but lags.

Which measurement structure? Two options, one choice

A All sensors as measurements

- A simple kinematic model propagates the state, and every sensor — including accelerometer and gyroscope — is treated as a measurement corrected in the update step.
- Conceptually simple, but it forces the process model to predict body specific force and angular rate — quantities set by thrust and un-modelled control action.
- The model must then invent a fictitious 'constant acceleration' assumption and fights it continuously.

B IMU as input — the strapdown INS/GPS structure

- Accelerometer and gyroscope become the input u that drives the prediction; GPS position, GPS velocity, barometric altitude and magnetometer yaw are the measurements that correct inertial drift.
- This mirrors how real flight controllers operate and matches the physics cleanly.

Option B was chosen — for three specific reasons

It avoids the fictitious process model of Option A · it places the IMU's noise where it physically enters, through the input and therefore into Q · and it lets the datasheet-derived GPS and magnetometer noise define R directly.

$$x = [\,x, y, z, v_x, v_y, v_z, \phi, \theta, \psi\,]^T \qquad u = [\,a_x, a_y, a_z, p, q, r\,]^T \qquad \textit{nine states, seven measurements}$$

Implementation — the two MATLAB functions

State-transition function f(x, u) — where all the nonlinearity lives

```
function xk1 = ekfStateTransition(xk, u)
% xk = [px py pz vx vy vz phi theta psi]'
% u  = [ax ay az p q r]'
Ts = 0.005;                        % 200 Hz IMU rate
phi=xk(7); theta=xk(8); psi=xk(9);
a_b = u(1:3);  w_b = u(4:6);

R = [ cth*cps, sph*sth*cps-cph*sps, cph*sth*cps+sph*sps;
      cth*sps, sph*sth*sps+cph*cps, cph*sth*sps-sph*cps;
      -sth,    sph*cth,             cph*cth ];
g = [0; 0; -9.81];

T = [ 1, sph*tan(theta), cph*tan(theta);      % body rates
      0, cph,           -sph;                 %  -> Euler rates
      0, sph/cth,        cph/cth ];

xdot = [ xk(4:6);         % p_dot = v
         R*a_b + g;       % v_dot
         T*w_b ];         % Theta_dot
xk1 = xk + Ts*xdot;       % forward Euler
```

Measurement function h(x) — a pure selection of states

```
function y = ekfMeasurement(xk)
% GPS position (x,y,z=baro),
% GPS velocity, magnetometer yaw
y = [ xk(1:6); xk(9) ];
```

- Three physical relations are assembled: $\dot{p} = v$, $\dot{v} = R \cdot a_b + g$, $\Theta = T \cdot \omega_b$ — then advanced by forward Euler at the 200 Hz IMU rate.
- Process noise w is not added in f: with additive process noise the EKF block adds it internally.
- H is therefore the constant matrix $[\, I_6 \;\; 0_{6\times3} \,]$; row selecting ψ], 7 × 9 of ones and zeros. All the nonlinearity is in f, none in h.
- **Two consequences worth stating precisely:**
- The third measurement is labelled GPS position but is physically the barometer altitude — both report the same state z, so the filter simply receives a better-quality vertical measurement.
- Roll and pitch appear in no row of h, so they are never measured. They are observed only indirectly: an attitude error rotates the accelerometer, corrupts the velocity prediction, and GPS then corrects it.

Covariance settings for the seven-measurement filter

Matrix	States / measurements	Diagonal values (variance)
R (7 × 7)	GPS x, y; barometer z; GPS v_x, v_y, v_z; magnetometer yaw	[6.25, 6.25, 0.04, 2.5e-3, 2.5e-3, 2.5e-3, 1.2e-3]
Q (9 × 9)	position; velocity; attitude	[1e-6 ×3, 1e-2 ×3, 1.5e-6 ×3] (tuned)
P_0 (9 × 9)	initial uncertainty	[6.25, 6.25, 0.04, 2.5e-3 ×3, 3e-3, 3e-3, 1.2e-3]

R comes from the datasheets

6.25 on the horizontal GPS entries is exactly (2.5 m)² — the NEO-6M's stated CEP. 0.04 on the barometer entry reflects how much cleaner the pressure-derived altitude is.

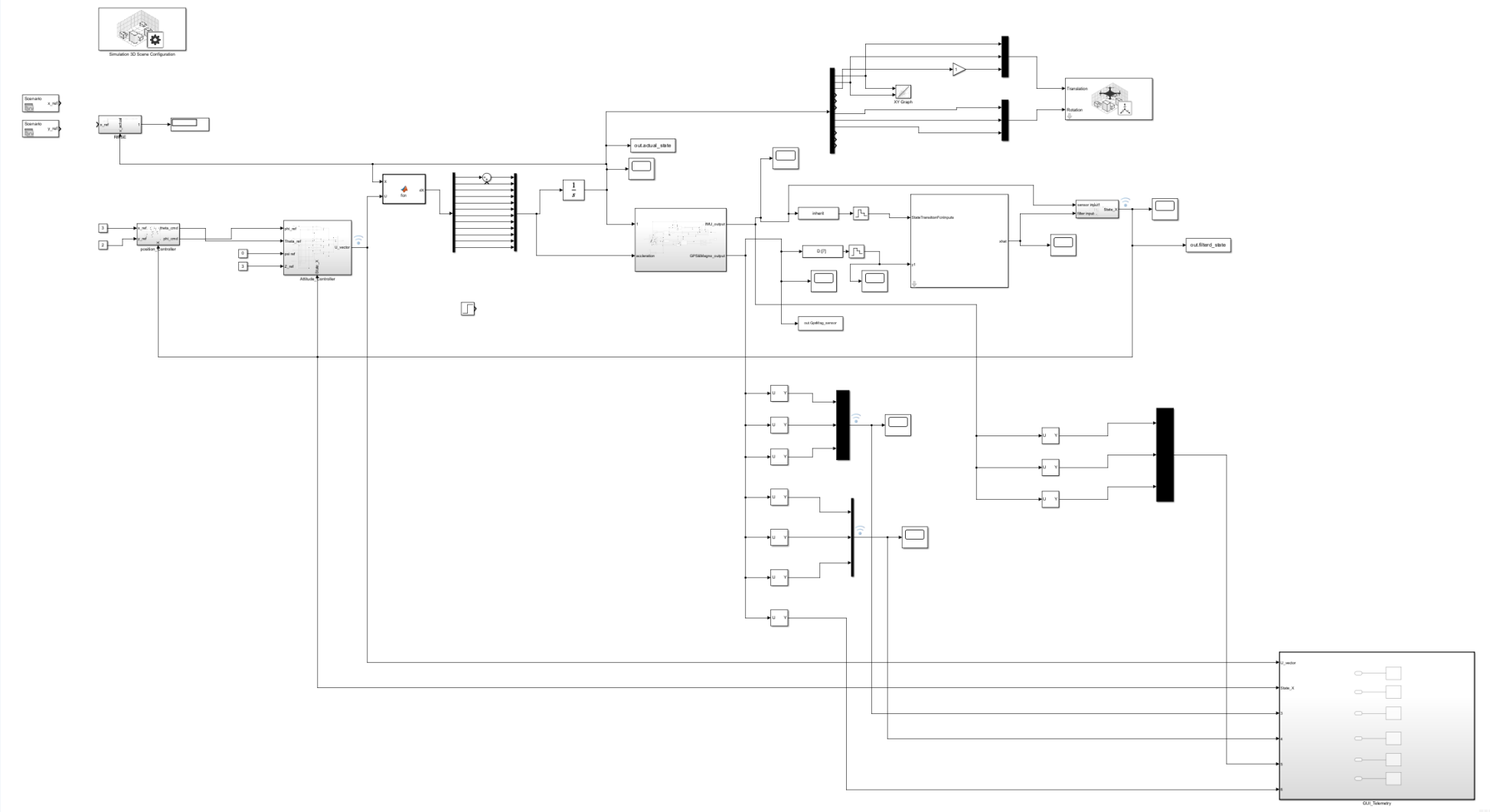
Q was seeded and then tuned

Velocity process noise is four orders of magnitude larger than position process noise, because velocity is where the accelerometer's noise physically enters the prediction.

P_0 mirrors R

At start-up the filter's uncertainty about a state is essentially the uncertainty of the sensor that measures it — so the initial covariance is set from the same figures.

The Entire Model:



Recovering the unmeasured states

Roll and pitch are reconstructed almost exactly — despite appearing in no row of the measurement model.

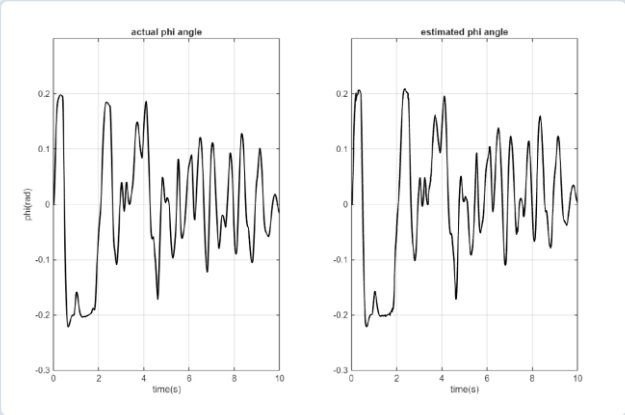


Figure 3.3 — roll: true ϕ (left) vs EKF estimate (right)

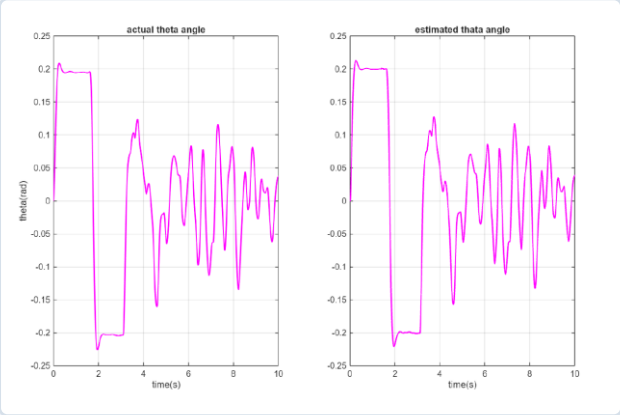


Figure 3.4 — pitch: true ϑ (left) vs EKF estimate (right)

The strongest single result of the chapter

- Aggressive, high-frequency manoeuvres of ±0.2 rad are tracked with only slight smoothing.
- No sensor in the vehicle reports roll or pitch. These traces are produced entirely by the estimator.

How is that possible?

- Through the indirect observability path: if the estimated attitude is wrong, the accelerometer specific force is rotated into the navigation frame incorrectly.
- That produces a wrong velocity prediction, which GPS velocity then contradicts. The resulting innovation corrects the attitude.
- The estimator — not a sensor — is what ultimately delivers ϕ and θ.

Filtering the horizontal position

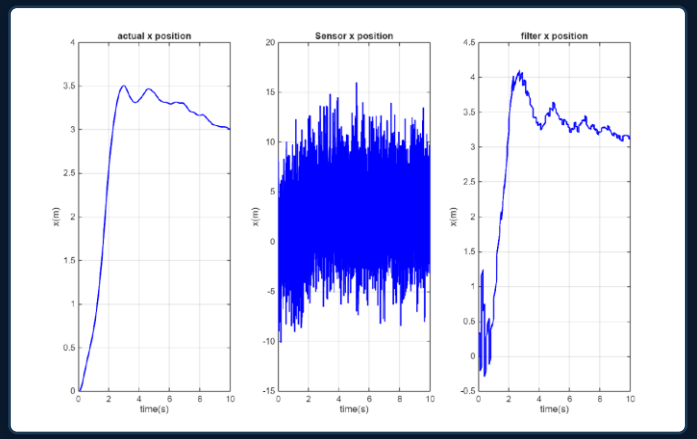


Figure 3.5 — x: true (left), raw GPS spanning ±10 m of noise (centre), EKF estimate (right)

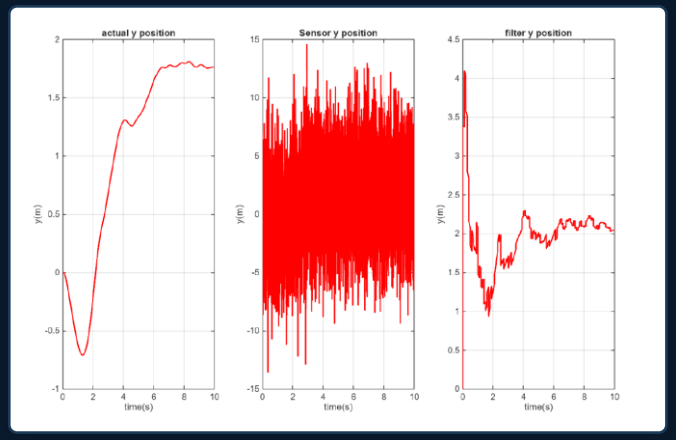


Figure 3.6 — y: true, raw sensor, EKF estimate

±10–15 m

raw GPS position noise — the instantaneous measurement is almost unusable

< 0.5 m

EKF estimate tracks the true trajectory to a few tenths of a metre

One to two orders of magnitude of noise reduction — converting an unusable sensor into a usable state.

Altitude and yaw — and what Chapter 3 established

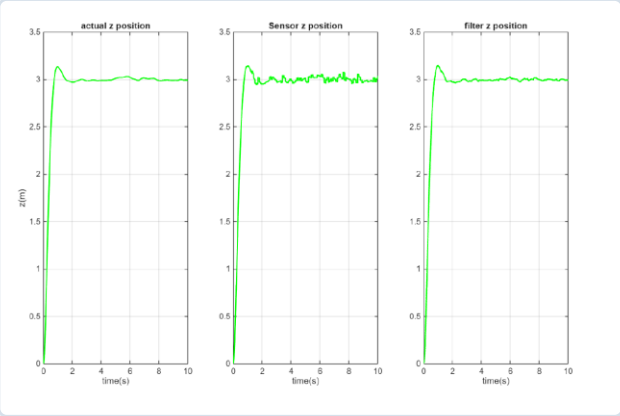


Figure 3.7 — z: true, barometer, EKF estimate, all on the 3 m set-point

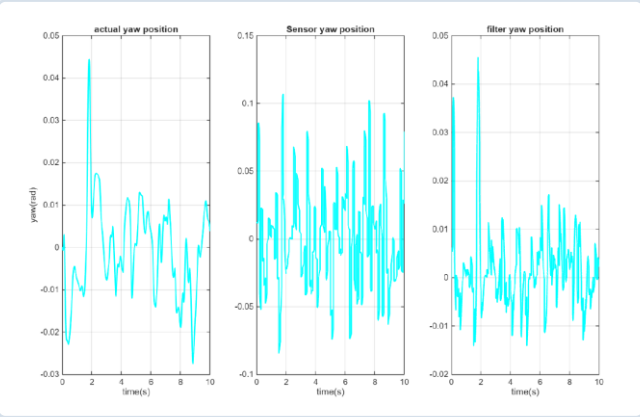


Figure 3.8 — yaw: true, magnetometer (±0.1 rad noise), EKF estimate

Altitude is the cleanest channel: the barometer-based measurement is already low-noise, so true, sensor and filtered traces all sit tightly on the 3 m set-point. The filter simply smooths residual ripple.

Yaw is the opposite case: the magnetometer measurement (±0.1 rad) is noisier than the small true yaw (±0.03 rad), yet the filter recovers the true heading — confirming the seventh measurement performs its intended anchoring role.

Chapter 3 in one card

- Four-source suite selected, with a rangefinder for the sub-1 m regime the barometer cannot serve.
- Critical quantities made redundant by fusing independent sources — though actuator redundancy remains beyond a quadcopter's reach.
- Two deficiencies motivated estimation: roll and pitch are measured by no sensor, and every measurement is heavily noisy.
- Both answered by a nine-state EKF in the IMU-as-input / GPS-and-magnetometer-as-measurement structure — which reconstructs the unmeasured attitude almost exactly and reduces raw position noise by one to two orders of magnitude.

04

The Conditioning Circuits

Six circuits, each designed from specification and validated in Proteus

- Power-supply conditioning — the foundation every other measurement rests on
- I^2C: pull-up sizing from the rise-time budget, and multi-device validation
- Logic-level translation and the UART/NMEA GPS interface
- The one genuinely analog problem: anti-aliasing filter plus zero-suppression and span

Digital or analog? — and the two protocols

Only one sensor needs classical conditioning

- The accelerometer, gyroscope, magnetometer and barometer each integrate transducer, amplifier and ADC on-chip and output digital data.
- Only the rangefinder produces an analog voltage requiring a filter-and-amplify chain. That split shapes the whole chapter.

Note on the target microcontroller

- The intended flight processor is the ESP32-WROOM-32 — 240 MHz dual-core with an FPU, needed to run the EKF in real time.
- It has no Proteus model, so the ATmega328P (Arduino Uno) was used for simulation instead.
- This is legitimate: the circuits exercise bus electrical and protocol layers, power, and level translation — none of which depend on the specific processor. The ESP32's I²C and UART peripherals implement the same standards.
- Where it differs — the ESP32's native 3.3 V logic removes the need for the level shifter — that is noted in the relevant section.

I²C — two wires, many devices

- SDA carries data, SCL the master's clock. Every device drives the lines open-drain: it can only pull low; external pull-ups return the line high.
- Two devices therefore cannot conflict electrically — the bus is low if anyone pulls it low.
- Each device has a unique 7-bit address and acknowledges by pulling SDA low on the ninth clock. That selective addressing lets three sensors share two wires.
- Because the pull-ups alone perform every low-to-high transition, their value directly governs rise time — the subject of Circuit 2.

USART — point-to-point, no shared clock

- Asynchronous: both ends agree a baud rate in advance (9600 for the NEO-6M) and each byte is framed by start and stop bits so the receiver re-synchronises every character.
- The GPS simply transmits ASCII NMEA 0183 continuously — no command, no addressing.
- Because timing is agreed rather than shared, correct operation depends critically on both ends running at the same effective clock — which becomes a practical problem in Circuit 5.

Power-supply conditioning

Built first, because every subsequent circuit depends on it.

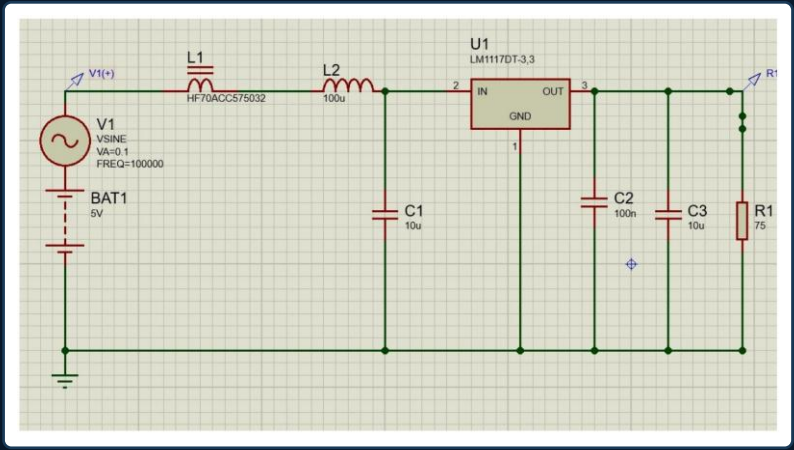


Figure 4.1 — ferrite bead, LC filter, LM1117 regulator, decoupling

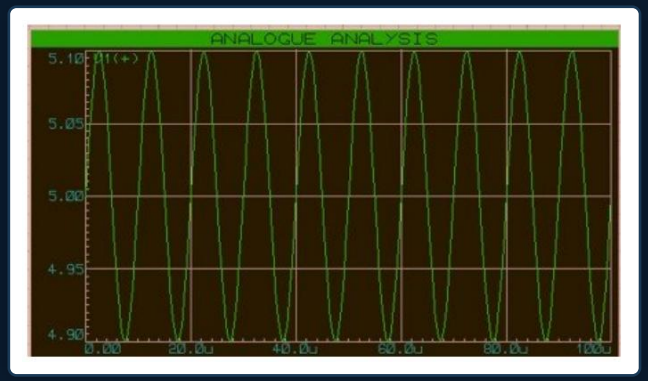


Figure 4.2a — noisy input rail

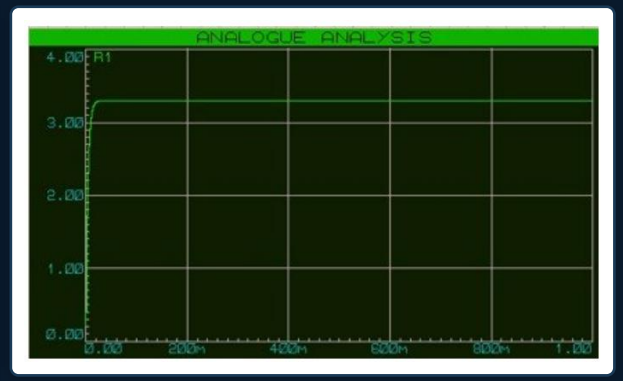


Figure 4.2b — clean regulated output

- **Why it was needed**
- Sensors are powered from the ESCs, whose switching regulators produce a rail heavily contaminated with high-frequency noise. Feeding that directly would corrupt every internal ADC conversion.
- **The chain**
- 5 V in → ferrite bead → LC low-pass → LM1117 3.3 V LDO → decoupling → sensor load.

$$f_c = 1/(2\pi\sqrt{LC}) = 1/(2\pi\sqrt{(100\mu H \cdot 10\mu F)}) \approx 5.0\ kHz$$

- ≈52 dB attenuation of the regulator's ~100 kHz switching ripple; the ferrite bead (≈300 Ω at 100 MHz) targets conducted EMI the LC filter cannot reach; the LM1117's ~60 dB PSRR removes the remainder.
- The load is modelled by 75 Ω — 3.3 V ÷ 44 mA, the summed draw of all four sensors. The GPS alone accounts for ~40 mA of that, dominating by 10×.

CH 4 · CIRCUIT 2

I^2C pull-up rise-time characterisation

The clearest instance in the project of designing from a specification rather than copying a common value.

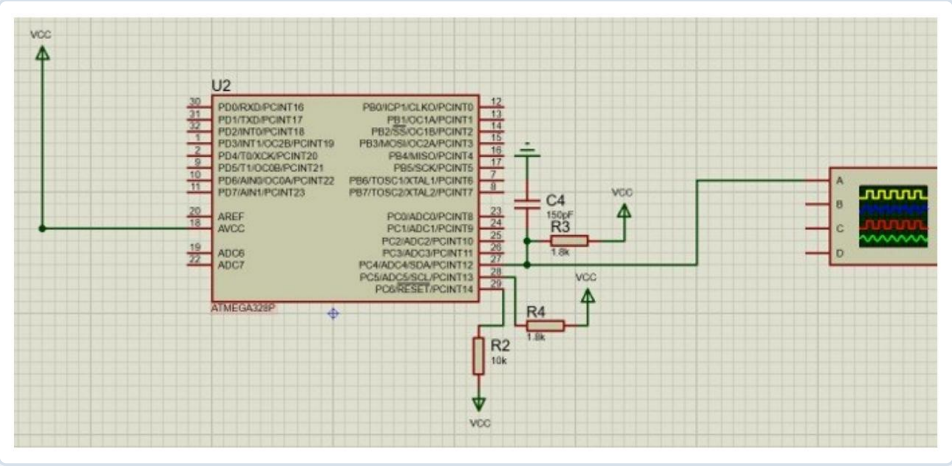


Figure 4.3 — open-drain driver, 1.8 kΩ pull-up, 150 pF bus-capacitance model

- **The failure mode being designed out**
- If the pull-ups are too large, high-going edges are too slow and a receiving device samples a bit before it has settled — corrupting data intermittently. That is the hardest kind of fault to diagnose.
- **The value is bracketed by two constraints**

$$R_{min} = (V_{dd} - V_{ol})/I_{ol} = (3.3 - 0.4)/3\ mA \approx 967\ \Omega$$
$$R_{max} = 300\ ns/(0.8473 \cdot 150\ pF) \approx 2.36\ k\Omega$$
$$\Longrightarrow \text{chosen } R = 1.8\ k\Omega,\quad t_r = 0.8473 \cdot R \cdot C$$

Pull-up	Calculated t_r (30–70 %)	400 kHz limit	Verdict
1.8 kΩ (chosen)	229 ns	≤ 300 ns	**PASS**
4.7 kΩ (common default)	597 ns	≤ 300 ns	**FAIL — 2× over**

A measurement note worth recording: the I^2C specification defines rise time between the 30 % and 70 % thresholds, not the more familiar 10–90 %. Using the wrong threshold definition would make a compliant bus appear to fail.

Multi-device I²C bus validation

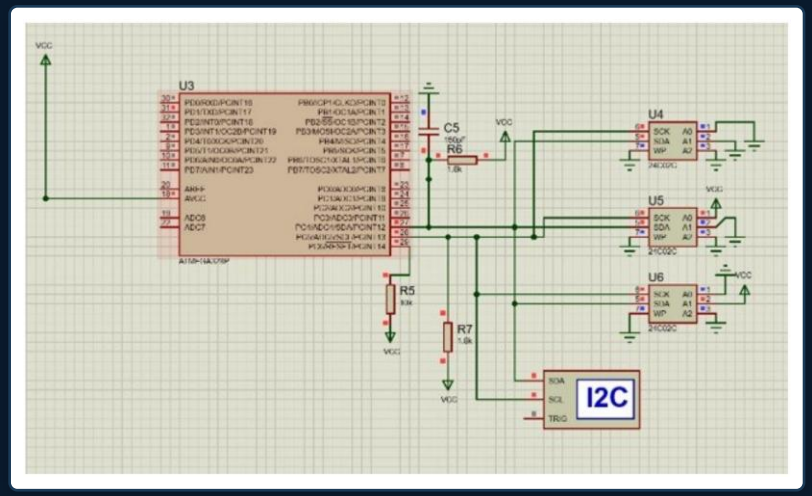


Figure 4.4 — three 24C02 EEPROMs at 0x50–0x52 with pull-ups and I²C debugger

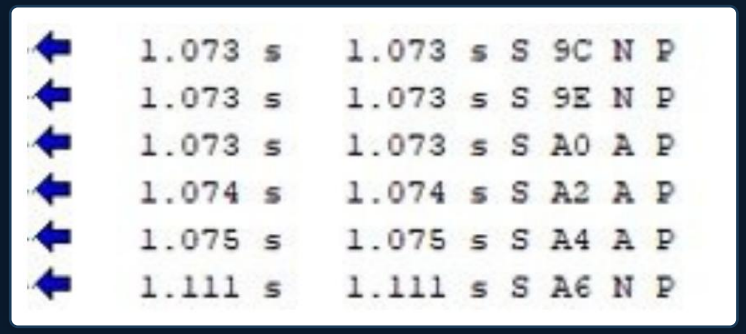


Figure 4.5 — the I²C scanner terminal output and debugger transaction trace

- **Circuit 2 proved the bus is electrically fast enough. Circuit 3 proves the protocol works — these are independent failure modes.**
- A bus can have perfect edges and still fail because of an address clash or a broken acknowledge.
- Proteus has no models of the actual sensor dies, so three 24C02 EEPROMs — real I²C slaves with configurable addresses — stand in at 0x50, 0x51 and 0x52.
- The substitution is legitimate because what is being tested is the bus, not the sensor: an EEPROM acknowledges at its address exactly as an MPU6050 acknowledges at 0x68. Only the data exchanged afterwards differs.

```
Wire.begin();
Wire.setClock(400000);   // the speed the
                         // pull-ups were designed for
for (byte addr = 1; addr < 127; addr++) {
  Wire.beginTransmission(addr);
  if (Wire.endTransmission() == 0)
    Serial.println(addr, HEX);   // it ACKed
}
```

Logic-level translation — 5 V ↔ 3.3 V

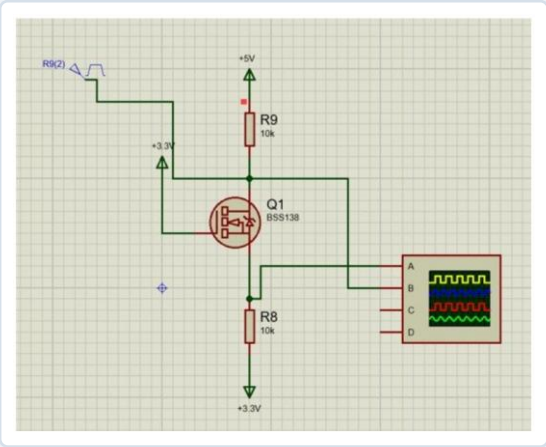


Figure 4.6 — BSS138 bidirectional level shifter

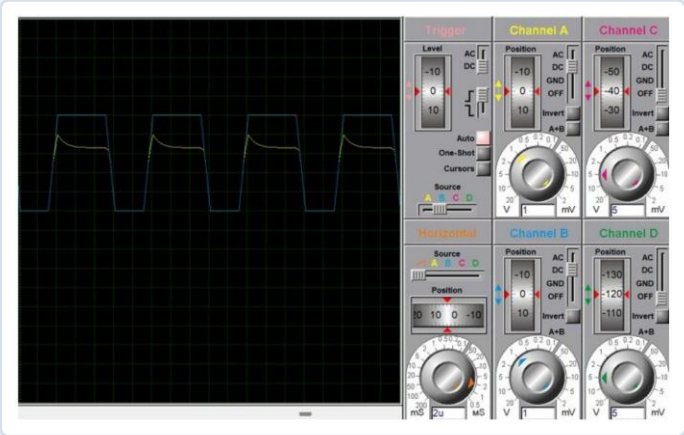


Figure 4.7 — both sides tracking in each translation direction

This circuit is partly an artefact of the 5 V simulation platform — the target ESP32 is natively 3.3 V — but it remains a genuine, reusable design, required whenever a 5 V and a 3.3 V I²C device must interoperate.

- **Why a resistor divider will not do**
- I²C is bidirectional and open-drain. A divider is one-directional and would load the bus. The standard solution is a single BSS138 MOSFET per line, with a pull-up to 3.3 V on the low side and to 5 V on the high side.

1 **Bus idle — both sides high**

Source sits at 3.3 V from its pull-up; gate is at 3.3 V, so V_GS = 0 and the FET is off. The drain floats up to 5 V via its own pull-up. Both sides high.

2 **A 3.3 V device pulls the low side down**

Source goes to ~0 V while the gate stays at 3.3 V. Now V_GS = 3.3 V, well above the ~1 V threshold, so the FET conducts and drags the drain down. The high side follows.

3 **The 5 V device pulls the high side down**

The FET is off — so how does the low side know? The body diode. Every N-MOSFET has an intrinsic source-to-drain diode; when the drain drops below the source it conducts, pulling the source to ~0.7 V. That drop makes V_GS exceed the threshold, the channel turns properly on, and both sides go to near 0 V.

GPS reception and LCD display — UART / NMEA

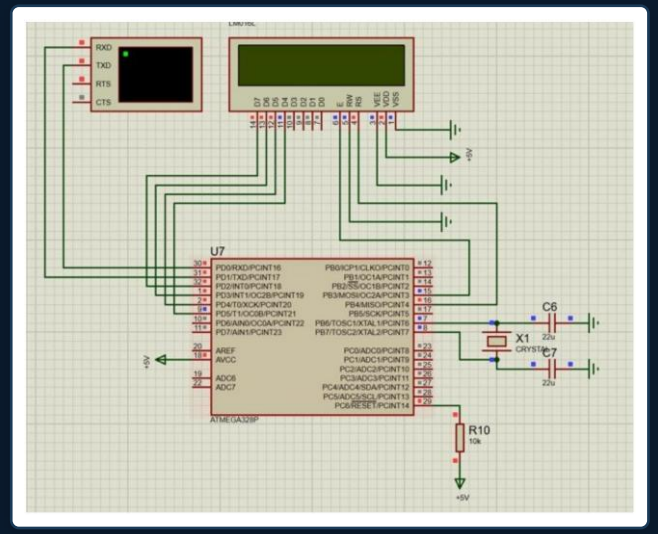
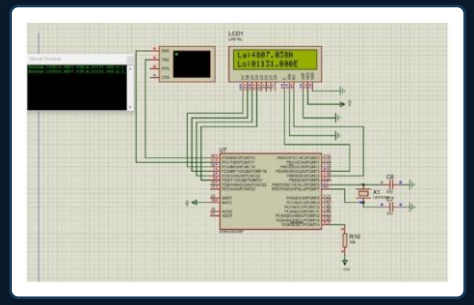
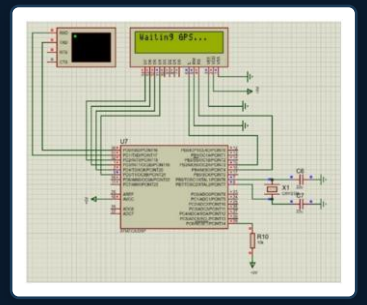


Figure 4.8 — microcontroller, 16×2 LCD in 4-bit mode, virtual terminal injecting NMEA



Parsed latitude / longitude



The no-fix warning case

- **The most complete of the digital-sensor circuits — nothing is approximated except the radio front-end itself.**
- The microcontroller receives NMEA at 9600 baud, parses the $GPGGA sentence for latitude and longitude, and displays them on a 16×2 LCD in 4-bit mode.
- Proteus has no NEO-6M model, so a virtual terminal injects real NMEA sentences — the exact ASCII a NEO-6M transmits. Protocol, baud rate, parsing and display are all validated end-to-end.
- A no-fix sentence (quality field 0) is also injected, confirming the firmware detects loss of fix and displays a warning rather than stale coordinates — the fault-detection logic that feeds the Chapter 3 fail-safes.

```
$GPGGA,123519,4807.038,N,01131.000,E,
        1,08,0.9,545.4,M,46.9,M,,*47

if (f[6] == "0") { lcd.print("NO FIX"); }
else { lcd.print("La:"); lcd.print(f[2]);
       lcd.print("Lo:"); lcd.print(f[4]); }
```

Zero suppression and span — using the whole ADC

- The MB1240 outputs Vcc/1024 per cm — about 4.88 mV/cm at 5 V.
- Over the landing regime of interest, 20–200 cm, the raw output spans only 97.6 mV to 976 mV: about 878 mV out of a 5 V ADC range — under 18 %.

$$Gain = 5.000\ V / (0.976 - 0.0976)\ V = 5.000/0.878 \approx 5.7$$

$$V_out = (V_in - 0.0976) \cdot 5.69$$

$$20\ cm \rightarrow 0\ V \quad 110\ cm \rightarrow 2.50\ V \quad 200\ cm \rightarrow 5.00\ V$$

- Implemented as a difference amplifier: R_1 = 10 kΩ, R_2 = 57 kΩ gives R_2/R_1 = 5.7.
- **Why a 2.5 V reference rather than the 5 V rail?**
- Because the motor load causes massive voltage sag and spikes on the 5 V rail, which would move the 0.0976 V reference and put the whole zero-and-span circuit in jeopardy. An LM4040-2.5 gives a precise, load-independent 2.5 V, divided down and buffered.
- A unity-gain buffer also precedes the amplifier so the sensor's output impedance does not corrupt the gain.
- **Resolution improves from ~1.0 cm/count to 0.18 cm/count on a 10-bit ADC — a factor of 5.69.**

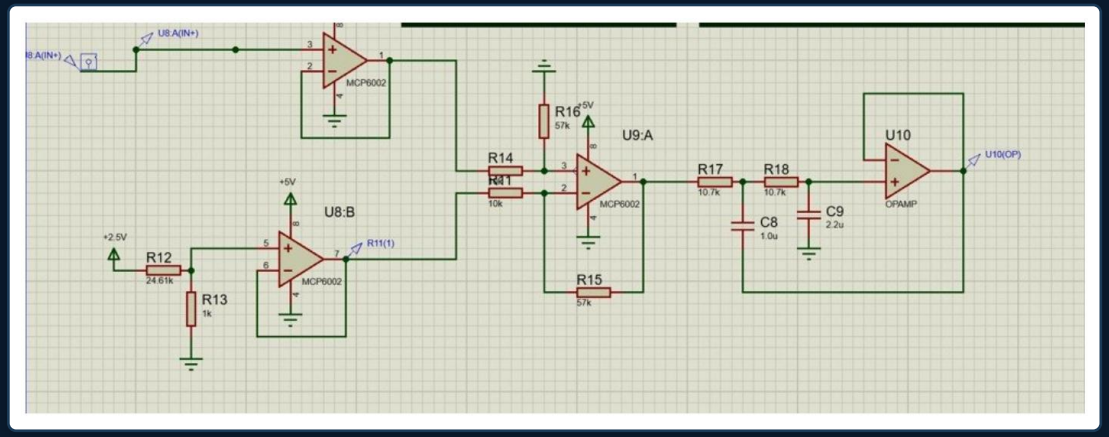
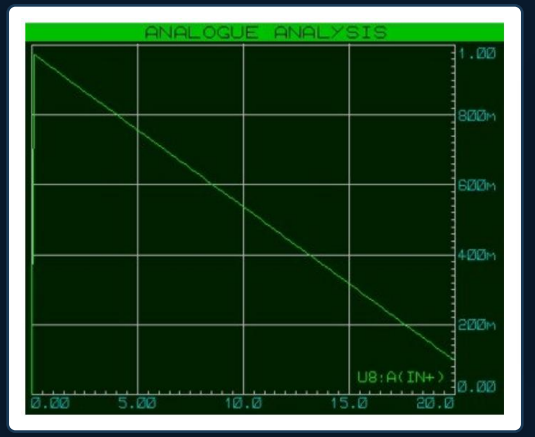
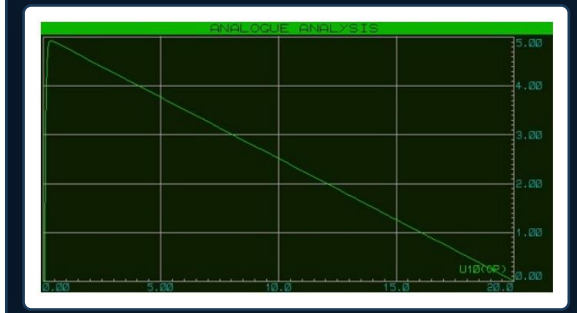


Figure 4.10 — buffer, difference amplifier, Sallen-Key filter



Raw 98–976 mV vs conditioned 0–5 V



−3 dB at the design cutoff

Anti-aliasing Butterworth filter design

$$H(s) = \omega_c^2 / (s^2 + (\omega_c/Q)\cdot s + \omega_c^2)$$

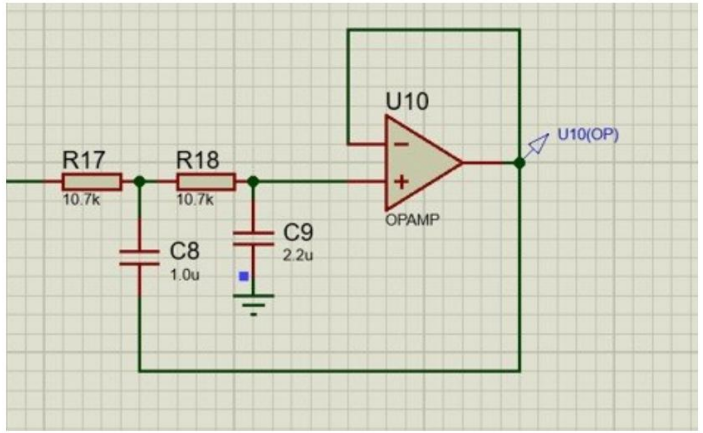
- Q = 1/√2 ≈ 0.707 defines the Butterworth case — maximally flat passband, no ripple and no peaking before cutoff, at the cost of a moderate transition slope.
- That flatness is exactly why Butterworth is preferred for a measurement channel: it does not distort the in-band signal. Above cutoff it rolls off at 40 dB/decade.

$$\omega_c = 1/\sqrt{(R_1 R_2 C_1 C_2)} \quad Q = \sqrt{(R_1 R_2 C_1 C_2)} / (C_1(R_1+R_2))$$
$$R_1 = R_2 = R \implies Q = \tfrac{1}{2}\sqrt{(C_2/C_1)} \implies C_2 = 2\cdot C_1$$

Setting the resistors equal reduces the Butterworth condition to a single capacitor ratio — the heart of the design.

Design	f_c	C_1 / C_2	R	Realised Q
A	5 Hz	1 µF / 2.2 µF	≈ 21.5 kΩ	≈ 0.74
B	10 Hz	1 µF / 2.2 µF	≈ 10.7 kΩ	≈ 0.74

Capacitors are chosen first (±10 % tolerance) and the resistor solved for (±1 % available). Q ≈ 0.74 rather than 0.707 comes from rounding C_2 to a standard 2.2 µF — ~0.1 dB of peaking, negligible. The design is tolerance-limited, not ratio-limited.



Simulated filter response

5 Hz or 10 Hz? A genuine trade-off

- Anti-alias rejection — halving the cutoff adds roughly 12 dB of rejection at Nyquist, so 5 Hz is the safer barrier.
- Group delay ≈ √2/(2π f_c) — about 45 ms at 5 Hz versus 22 ms at 10 Hz. The lower cutoff doubles the lag the altitude loop must tolerate as added phase.
- Signal fidelity — both sit comfortably above the ~1–2 Hz descent signal, so neither distorts the measurement. The distinction is purely rejection versus delay.
- 5 Hz is the default; 10 Hz is held as the lower-delay alternative.

CH 4 · SUMMARY

Six circuits, and a layered validation strategy

#	Circuit	Purpose	Key result
1	Power supply	Clean 3.3 V from a noisy rail	≈52 dB ripple rejection (LC) plus LDO
2	I²C pull-up	Size pull-ups from specification	1.8 kΩ → 229 ns (pass); 4.7 kΩ fails
3	Multi-device bus	Validate three-device addressing	3 slaves enumerated; ACK verified
4	Level shifter	5 V ↔ 3.3 V translation	Bidirectional transfer confirmed
5	GPS + LCD	UART/NMEA reception and display	Lat/lon parsed; loss-of-fix detected
6	Rangefinder	Analog zero/span plus filter	0–5 V span; Butterworth at 5/10 Hz

Turning a tooling limitation into a deliberate validation strategy

- Proteus has no behavioural models for the MPU6050, MS5611, HMC5883L or NEO-6M — they are digital sensors with integrated ADCs and no published models.
- So the digital interfaces were validated in three separate layers: ELECTRICAL (pull-up sizing against the rise-time specification, supply noise rejection, level translation) · PROTOCOL (multi-device I²C addressing and acknowledge captured with the debugger; UART/NMEA parsing validated end-to-end) · DATA (sensor values represented by the validated models of Chapter 3).
- Together with the one genuinely analog circuit, this establishes that every sensor in the suite can be interfaced cleanly, safely, and at the correct signal integrity.

05

Selecting the Required Actuators

The electro-mechanical chain that turns numbers into thrust

- Brushless DC motors — A2212 1000KV
- Propellers and the aerodynamic linkage — 1045 / 1045R
- Electronic speed controllers — SimonK / BLHeli 30 A
- The motor mixer: virtual efforts to physical rotor speeds

The actuation chain

BLDC motors — A2212 1000KV

- Electronic commutation rather than mechanical brushes: much higher power-to-weight ratio, better efficiency, longer life.
- 1000KV means ~1000 RPM per volt at no load. On a 3S LiPo at 11.1 V that is ≈11,100 RPM no-load; under propeller load it drops.
- RPM = KV × V_supply. The 1000KV rating balances torque against angular velocity — fast response to step commands without excessive current draw.

Propellers — 1045 and 1045R

- Thrust is highly nonlinear: $T = k_f \omega^2$. Diameter and pitch set k_f, and therefore total thrust, current draw and stability.
- 10 inch diameter gives a large disc area — substantial thrust at lower RPM, efficient for stable hover. The 4.5 inch pitch prevents stalling and keeps airflow predictable.
- Two standard (tractor) and two reverse (pusher, 'R') propellers, mounted diagonally opposite, so the reactive drag torques cancel in level hover and there is no unwanted yaw.

ESCs — SimonK / BLHeli 30 A

- A microcontroller cannot drive a 3-phase BLDC directly. The ESC takes the low-voltage PWM command and adjusts the timing and power of the 3-phase AC applied to the stator.
- Multirotor firmware bypasses the heavy internal filtering and low-voltage cutoffs of aeroplane ESCs, giving linear throttle curves and high-refresh input (up to 400 Hz, or digital protocols such as DShot) — drastically reducing the actuator time constant.
- 30 A gives generous margin over the A2212's typical 12–15 A peak draw, ensuring thermal stability and preventing mid-air shutdown.

The motor mixer is not hardware, but it is arguably the most critical executive block: it takes the four virtual efforts U_1–U_4 and decomposes them into four squared-speed commands — thrust distributed equally, roll as a left/right differential, pitch as a front/rear differential, yaw as a CW/CCW differential that turns the chassis without altering total lift.

06

Graphical User Interface

A ground-control station that watches the simulation and the hardware at once

- An atomic subsystem and an execution-event listener
- MATLAB App Designer: Controller tab and Sensor tab
- Bridging a 0.005 s solver step to a 20 Hz UI refresh
- Run_CoSimulation — the master launch sequencer

Listener architecture and the App Designer backend

Why the telemetry block had to be atomic

- To bridge the numerical simulation with an interactive GUI without interrupting the solver loop, a dedicated GUI_Telemetry block was created and configured as an atomic subsystem.
- Making a subsystem atomic forces Simulink to execute it as a single, indivisible computational unit each time step.
- That is what allows an execution event listener to be attached to its PostOutputs phase — guaranteeing the listener fires exactly once per timestep, with all input signals fully computed and stable.

Two tabs, deliberately compartmentalised

- Controller tab — control effort (thrust, roll, pitch, yaw), state position, state velocity, state attitude.
- Sensor tab — GPS position, GPS velocity, IMU acceleration, magnetometer yaw.
- Public buffer properties grow dynamically to hold the history each plot draws; private properties hold the serial device, hardware timer, current time and the animated line object.

```
% --- rate bridging in updateGUI ---
app.FastCounter = app.FastCounter + 1;

% solver runs at 0.005 s (200 Hz);
% UI is refreshed every 10th step (20 Hz)
if mod(app.FastCounter, 10) == 0
    app.TimeBuffer = [app.TimeBuffer, currentTime];
    app.StatePosBuffer = ...
        [app.StatePosBuffer; currentStates(1:3)'];
    app.StateVelBuffer = ...
        [app.StateVelBuffer; currentStates(7:9)'];
    plot(app.UIAxes, app.TimeBuffer, app.UBuffer);
    drawnow limitrate;   % render now, drop frames
end                      % if we fall behind
```

- Indices 1–3 of the 12-state vector are XYZ position, 7–9 are XYZ velocity, and index 6 is yaw.
- drawnow limitrate forces MATLAB to render immediately while allowing it to drop frames if the rendering pipeline falls behind the data stream — without freezing Simulink.

Run_CoSimulation — the master launch sequencer

1

A questdlg pop-up asks the user to choose 'Simulation Only', 'Hardware Only' or 'Run Both'.

2

The choice is pushed to the base workspace with assignin as ModeSelection, so the GUI can read it during boot.

3

An isempty check aborts with an error if the dialog is closed without a choice.

4

The App Designer interface is instantiated by calling DroneGCS.

5

load_system loads the Simulink model Main_with_GuiG into memory in the background.

6

The exact block path is composed: [modelName, '/GUI_Telemetry'].

7

Unless 'Hardware Only' was chosen, set_param issues SimulationCommand → start.

8

A while loop polls SimulationStatus in 50 ms increments until the engine has finished compiling and is officially running.

9

Only then is add_exec_event_listener attached to the PostOutputs phase, passing block data, event data and the app object into updateGUI.

On the hardware side, startupFcn opens a serialport on COM9 at 115200 baud, prepares a high-speed animatedline, and starts a 50 Hz background timer calling ReadHardwareData — all wrapped in try-catch so the app does not crash if the ESP32 is unplugged.

07

Hardware Implementation

From Proteus to a working ESP32 data-acquisition chain

- The physical conditioning chain and voltage divider
- ESP32 firmware: 12-bit ADC, linearisation, three output channels
- Proteus co-simulation through the COMPIM block
- Live telemetry over USB serial and Bluetooth

The physical conditioning chain

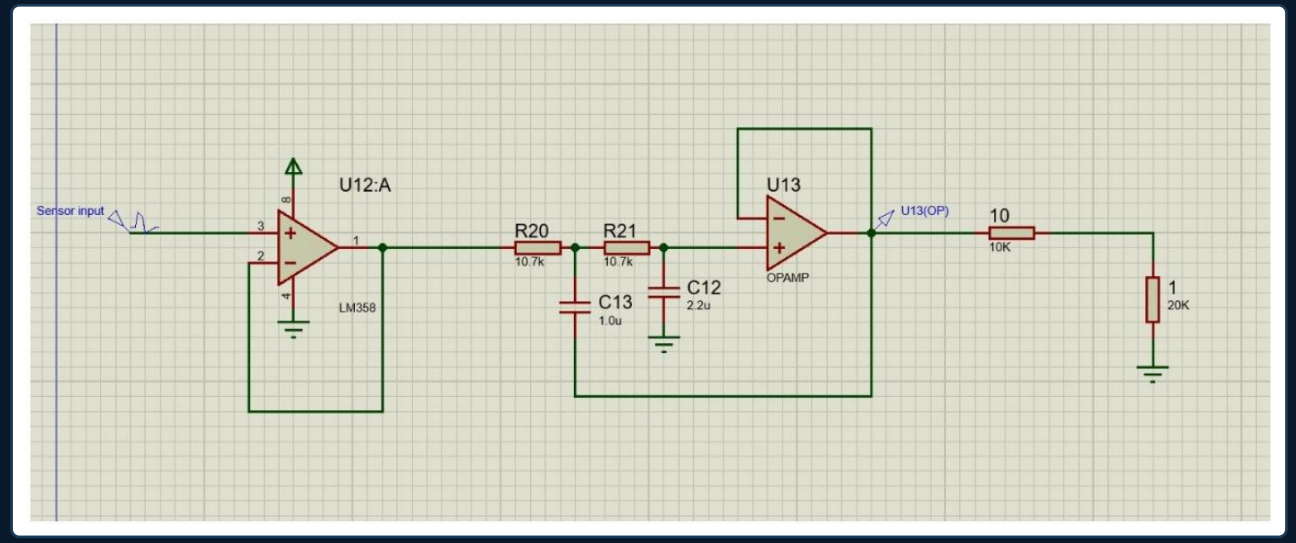


Figure 7.1 — the analog conditioning circuit and voltage divider

- Note the difference from the Chapter 4 design: because the sourced sensor is the nonlinear GP2Y0A21YK rather than the MB1240, the zero-suppression-and-span stage is dropped in favour of a firmware linearisation, and a protective divider is added instead.

1. **MB102 supply**
   Regulates the 12 V adapter down to a 5 V or 3.3 V DC rail.

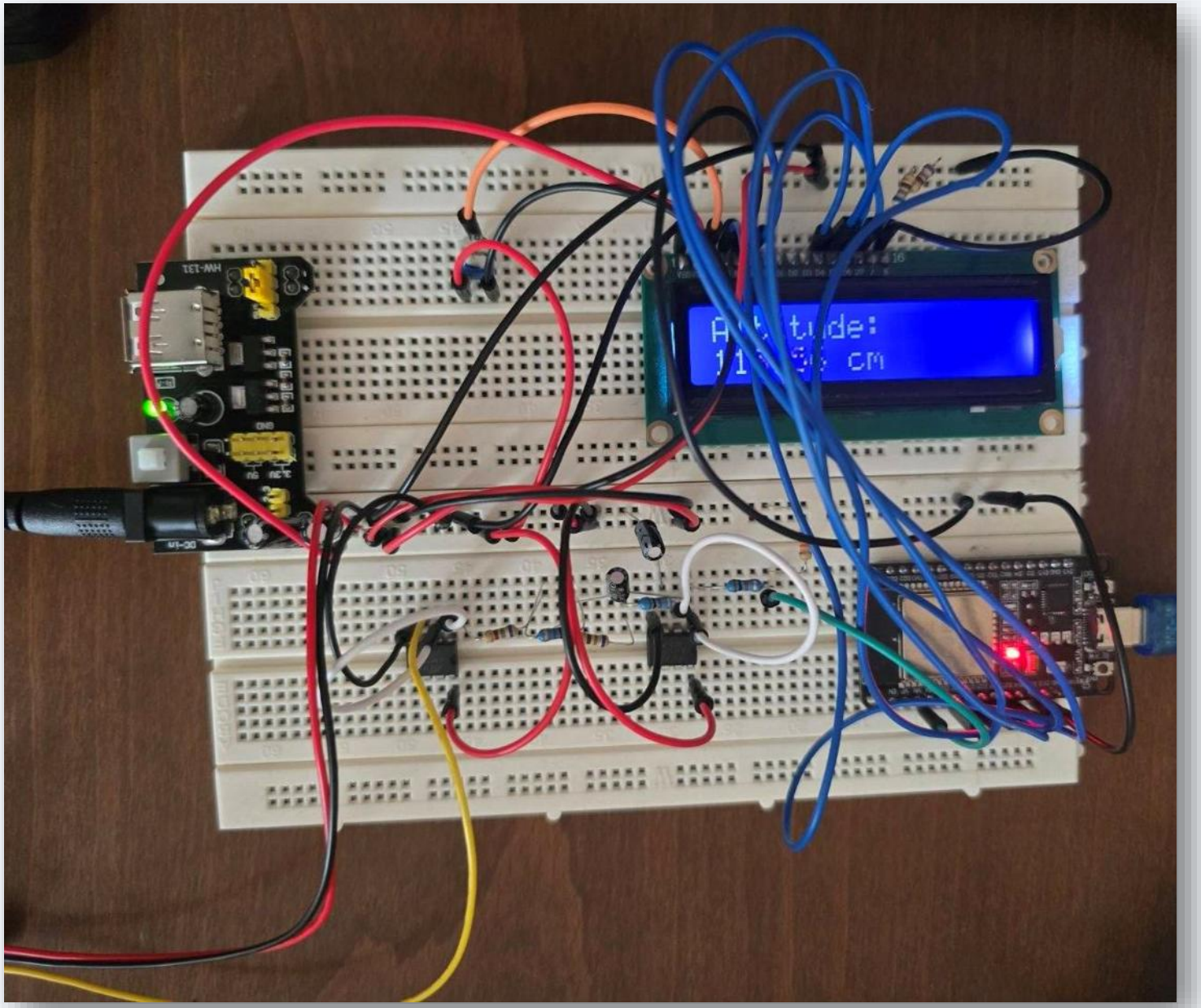
2. **Buffer amplifier**
   An LM358 in unity-gain feedback transfers the sensor voltage without loading it.

3. **Butterworth filter**
   The Chapter 4 design, filtering noise above 10 Hz.

4. **Voltage divider**
   Scales the filter output by 2/3 — ESP32 input pins must not exceed 3.3 V, so this guarantees the limit is never crossed.

5. **ESP32 analog input**
   The divided signal enters the ESP32's analog pin (D34) and its internal 12-bit ADC.

The circuit:



ESP32 firmware and the three output channels

```
const int analogPin = 34;      // from the voltage divider
const int resolution = 4095;   // 12-bit ADC maximum
const float refVoltage = 3.3;  // ESP32 logic level

void loop() {
  int rawADC = analogRead(analogPin);
  if (rawADC == 0) rawADC = 1;        // guard divide-by-zero

  float measuredVoltage =
        (rawADC / (float)resolution) * refVoltage;

  // reverse the 2/3 divider
  float trueSensorVoltage = measuredVoltage * 1.5;

  // linearise:  d = 27.86 * V^(-1.15) - 0.42
  float distance =
        27.86 * pow(trueSensorVoltage, -1.15) - 0.42;

  lcd.print(distance);          // physical 16x2 LCD
  Serial.println(distance);     // USB to MATLAB / Proteus
  SerialBT.println(distance);   // Bluetooth to any phone
  delay(20);                    // 50 Hz loop
}
```

Physical LCD

A 16×2 character display driven directly from the ESP32 shows live altitude at the vehicle — the sensor read-out required by the project brief.

USB serial at 115200

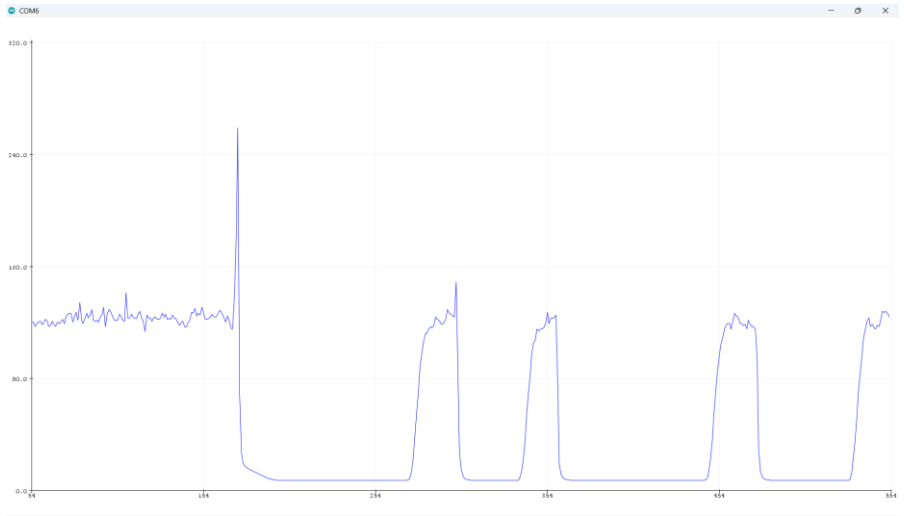
Feeds the Arduino IDE serial plotter, the dedicated MATLAB GUI tab, and — through a COMPIM block — the Proteus simulation.

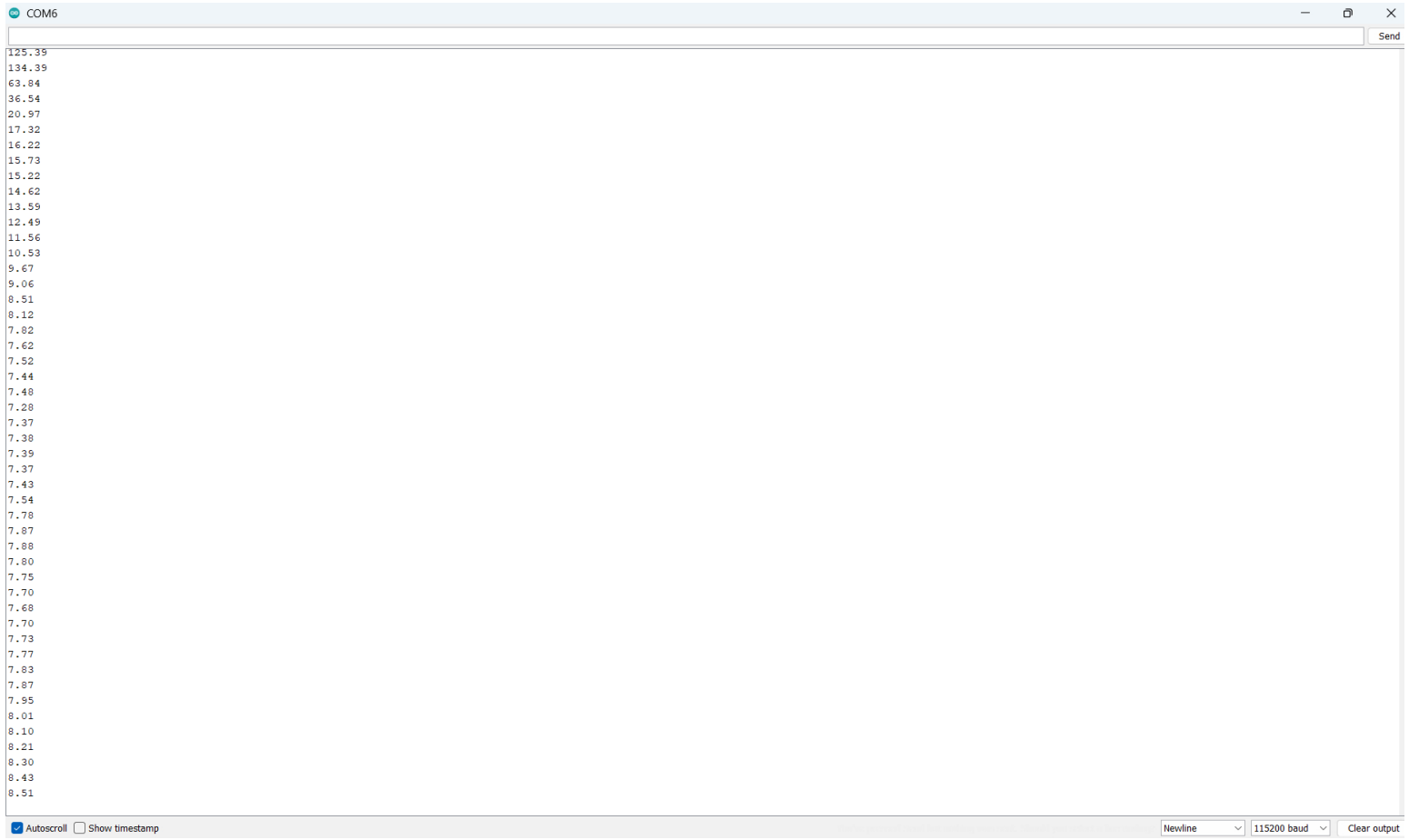
Bluetooth Serial

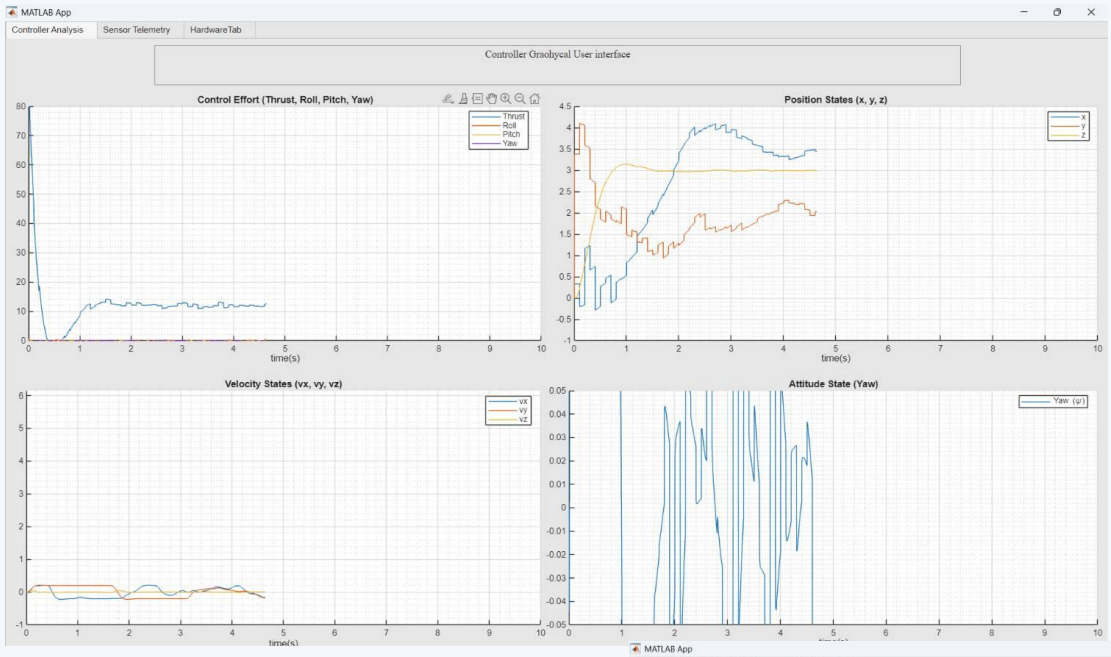
The device advertises as 'Drone_Telemetry'. Any mobile phone with a serial Bluetooth terminal app can receive the live stream — a genuine wireless telemetry link.

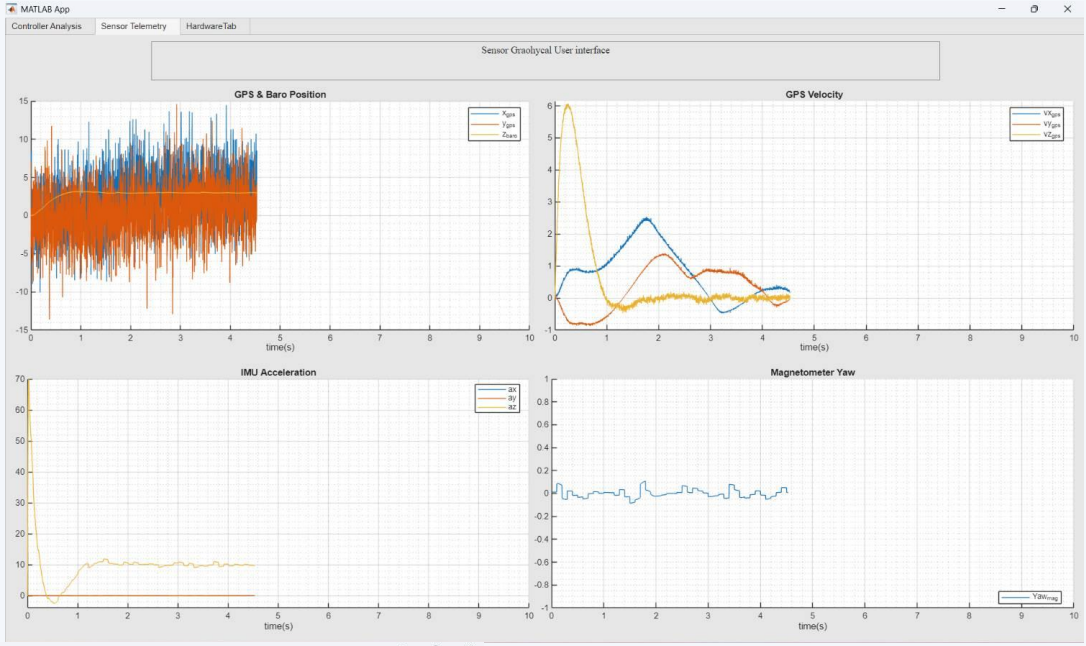
The firmware is the last link in the chain: raw analog voltage in, calibrated engineering units out, distributed over three independent channels.

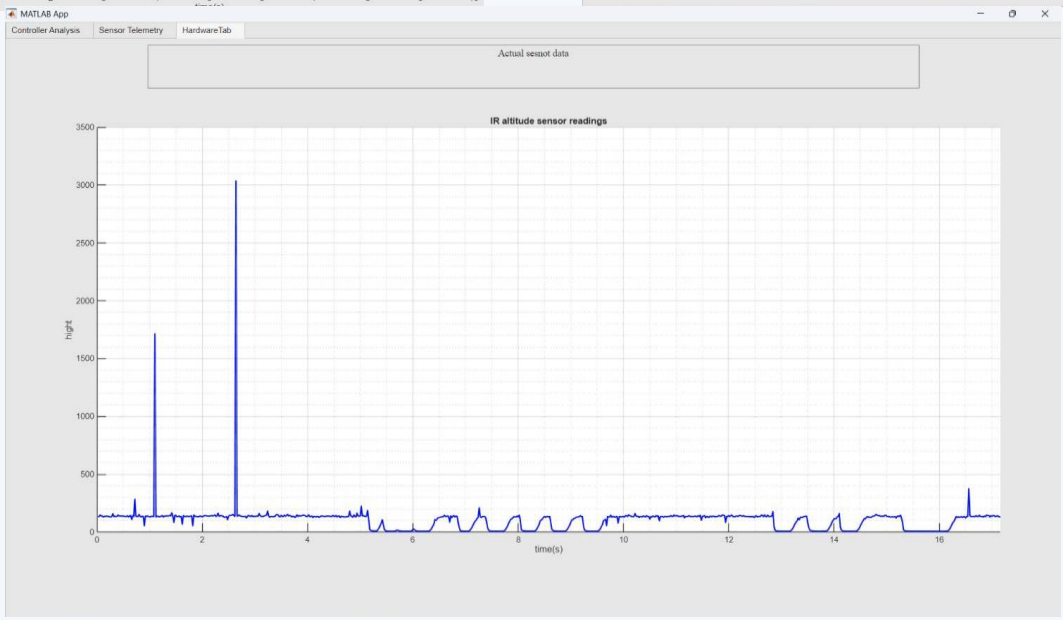
The results:


COM6
320.0
240.0
160.0
80.0
0.0
54
154
254
354
454
554


COM6
Send
125.39
134.39
63.84
36.54
20.97
17.32
16.22
15.73
15.22
14.62
13.59
12.49
11.56
10.53
9.67
9.06
8.51
8.12
7.82
7.62
7.52
7.44
7.48
7.28
7.37
7.38
7.39
7.37
7.43
7.54
7.78
7.87
7.88
7.80
7.75
7.70
7.68
7.70
7.73
7.77
7.83
7.87
7.95
8.01
8.10
8.21
8.30
8.43
8.51
Autoscroll
Show timestamp
Newline
115200 baud
Clear output


MATLAB App
Controller Analysis
Sensor Telemetry
HardwareTab
Controller Graohycal User interface
Control Effort (Thrust, Roll, Pitch, Yaw)
Thrust
Roll
Pitch
Yaw
time(s)
Position States (x, y, z)
x
y
z
time(s)
Velocity States (vx, vy, vz)
vx
vy
vz
time(s)
Attitude State (Yaw)
Yaw (ψ)


MATLAB App
Controller Analysis
Sensor Telemetry
HardwareTab
Sensor Graohycal User interface
GPS & Baro Position
time(s)
GPS Velocity
time(s)
IMU Acceleration
ax
ay
az
time(s)
Magnetometer Yaw
time(s)


MATLAB App
Controller Analysis
Sensor Telemetry
HardwareTab
Actual sensor data
IR altitude sensor readings
hight
time(s)

Proteus co-simulation and honest observations

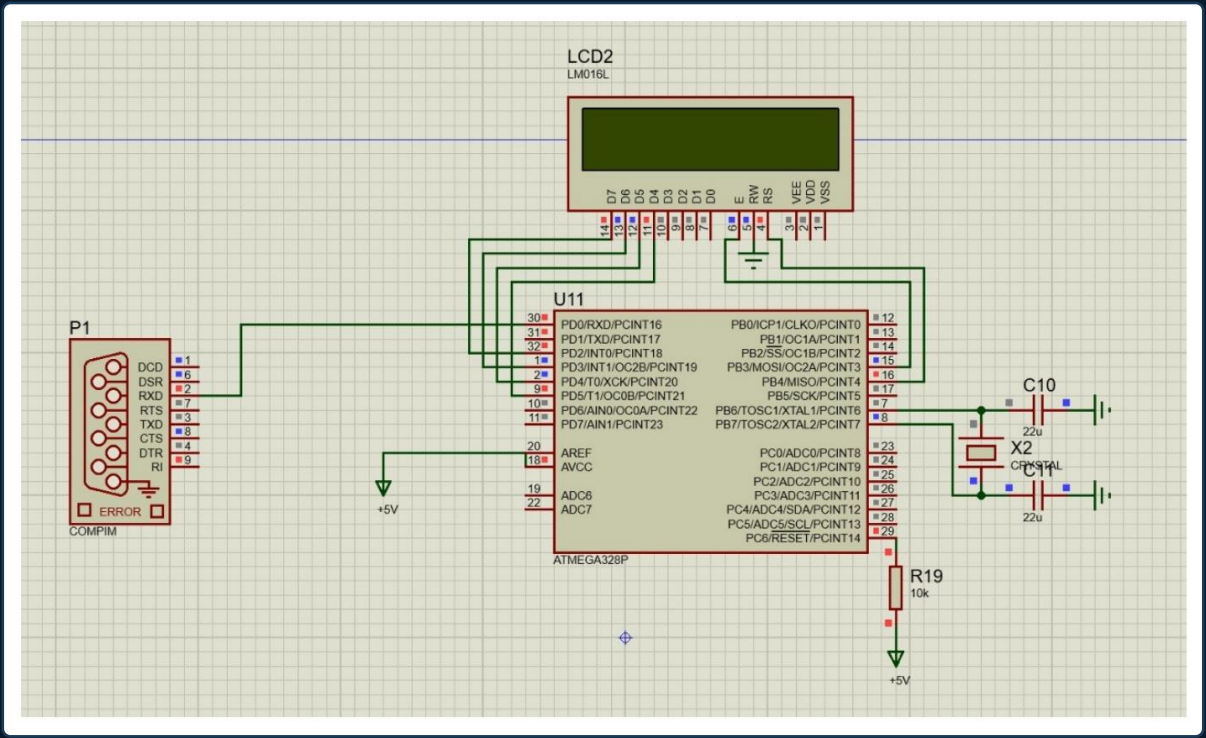


Figure 7.2 — the Proteus circuit using the COMPIM block

How the co-simulation works

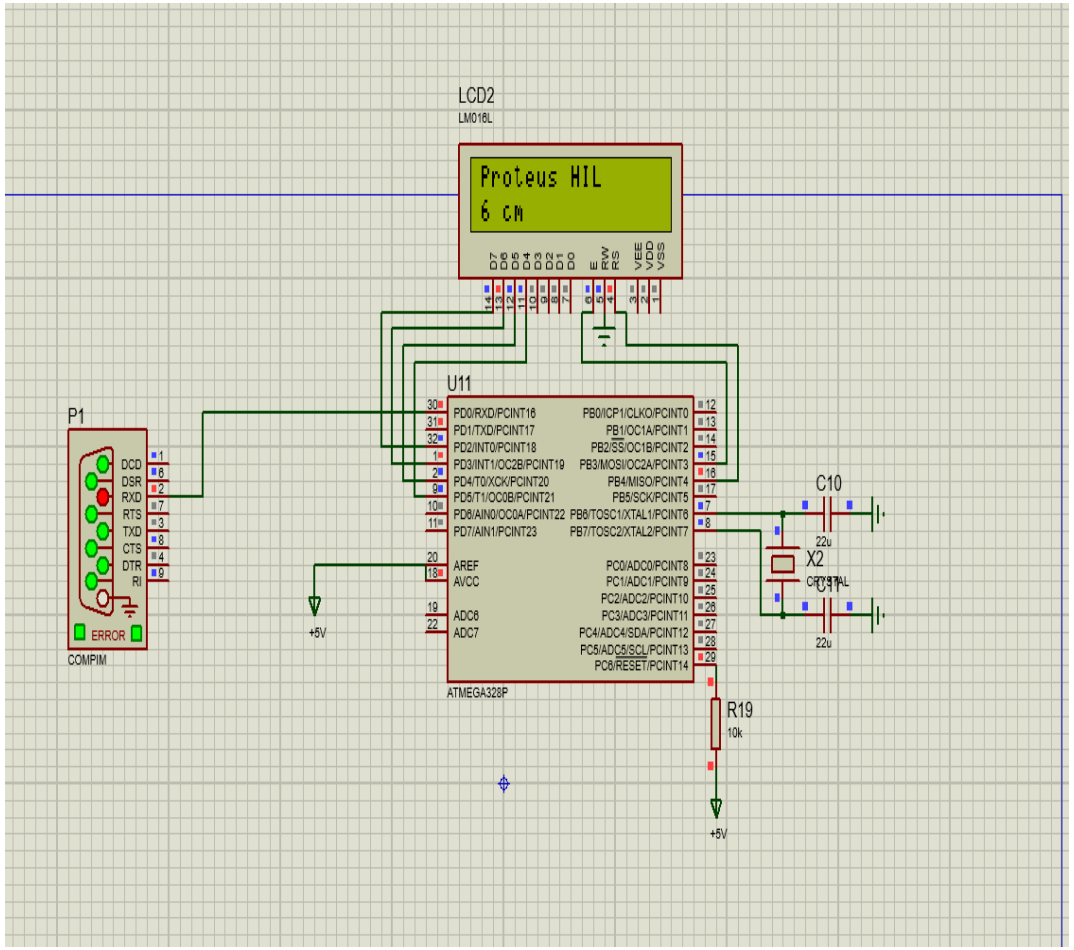
- A COMPIM block receives the real sensor data over the serial port. It is wired to the RXD pin of the simulated microcontroller.
- The simulated micro then acts as a translator — reading the live data and commanding the LCD to display it.

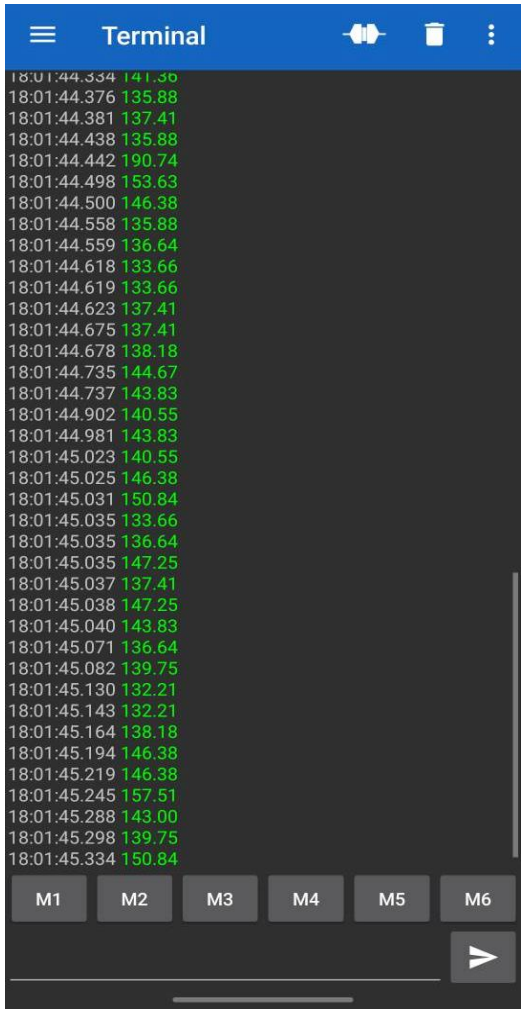
Observation 1 — Proteus runs slower than real time

At least on our laptops, substantially so. The LCD in Proteus may still be showing the very first sample while the microcontroller has already transmitted thousands. Worth knowing before you demonstrate it.

Observation 2 — the MATLAB plot is heavy

The dedicated GUI tab plots hardware data slowly, because MATLAB's plot function is inherently expensive and the data arrives fast. animatedline and drawnow limitrate mitigate it but do not eliminate it.


LCD2
LM016L
Proteus HIL
6 cm
U11
ATMEGA328P
P1
COMPIM
ERROR
C10
22u
X2
C11
22u
R19
10k
+5V
+5V


Terminal
18:01:44.376 135.88
18:01:44.381 137.41
18:01:44.438 135.88
18:01:44.442 190.74
18:01:44.498 153.63
18:01:44.500 146.38
18:01:44.558 135.88
18:01:44.559 136.64
18:01:44.618 133.66
18:01:44.619 133.66
18:01:44.623 137.41
18:01:44.675 137.41
18:01:44.678 138.18
18:01:44.735 144.67
18:01:44.737 143.83
18:01:44.902 140.55
18:01:44.981 143.83
18:01:45.023 140.55
18:01:45.025 146.38
18:01:45.031 150.84
18:01:45.035 133.66
18:01:45.035 136.64
18:01:45.035 147.25
18:01:45.037 137.41
18:01:45.038 147.25
18:01:45.040 143.83
18:01:45.071 136.64
18:01:45.082 139.75
18:01:45.130 132.21
18:01:45.143 132.21
18:01:45.164 138.18
18:01:45.194 146.38
18:01:45.219 146.38
18:01:45.245 157.51
18:01:45.288 143.00
18:01:45.298 139.75
18:01:45.334 150.84
M1
M2
M3
M4
M5
M6

Conclusion — what was delivered

1 Mathematical foundation

Newton–Euler equations and Euler angles governing the 6-DOF rigid-body dynamics, with the mixer matrix derived for the standard cross configuration.

2 Cascaded control

Inner-loop PD stabilising attitude and altitude to a 5 % overshoot specification; outer-loop PID tracking position and rejecting wind, tuned by RMSE to $K_p = 9$, $K_d = 5.4$, $K_I = 2$.

3 State estimation

A nine-state EKF fusing 200 Hz inertial data with 10 Hz absolute measurements — reconstructing roll and pitch that no sensor measures, and cutting positional noise by one to two orders of magnitude.

4 Signal integrity

Six conditioning circuits validated in Proteus: power regulation, I^2C pull-up sizing from specification, level translation, UART/NMEA reception, and a Butterworth plus zero-span analog chain.

5 Actuation

A2212 1000KV motors, 1045/1045R propellers and 30 A ESCs, mathematically linked to the controllers through the motor mixer.

6 Hardware & interface

A MATLAB App Designer GUI for real-time telemetry, and an ESP32 implementation delivering calibrated flight data over LCD, USB serial and Bluetooth.

An integrated approach producing a stable, fault-tolerant and fully observable autonomous flight platform.

Thank you

Questions welcome — on the control design, the estimator, and the circuits.

Amirkabir University of Technology · Instrumentation & Control · Dr. Ahmad Afshar